

การประยุกต์ใช้ CHECKPOINT/RESTORE ด้วย CRIU สำหรับระบบการประมวลผลแบบจำลองโมเลกุลแบบกระจาย

โดย

นายณัฐนันต์ โพธิ์ประดิษฐ์

โครงงานพิเศษนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
วิทยาศาสตรบัณฑิต
สาขาวิชาวิทยาการคอมพิวเตอร์
คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยธรรมศาสตร์
ปีการศึกษา 2568
ลิขสิทธิ์ของมหาวิทยาลัยธรรมศาสตร์

การประยุกต์ใช้ CHECKPOINT/RESTORE ด้วย CRIU สำหรับระบบการประมวลผลแบบจำลองโมเลกุลแบบกระจาย

โดย

นายณัฐนันต์ โพธิ์ประดิษฐ์

โครงงานพิเศษนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
วิทยาศาสตรบัณฑิต
สาขาวิชาวิทยาการคอมพิวเตอร์
คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยธรรมศาสตร์
ปีการศึกษา 2568

ลิขสิทธิ์ของมหาวิทยาลัยธรรมศาสตร์

APPLICATION OF CHECKPOINT/RESTORE USING CRIU FOR DISTRIBUTED MOLECULAR SIMULATION SYSTEMS

BY

MR NATANAN PHOPRADIT

A FINAL-YEAR PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR THE DEGREE OF BACHELOR OF SCIENCE
COMPUTER SCIENCE
FACULTY OF SCIENCE AND TECHNOLOGY
THAMMASAT UNIVERSITY
ACADEMIC YEAR 2025

COPYRIGHT OF THAMMASAT UNIVERSITY

มหาวิทยาลัยธรรมศาสตร์
คณะวิทยาศาสตร์และเทคโนโลยี

รายงานโครงงานพิเศษ

ของ

นายณัฐนันต์ โพธิ์ประดิษฐ์

เรื่อง

การประยุกต์ใช้ CHECKPOINT/RESTORE ด้วย CRIU สำหรับระบบการประมวลผลแบบจำลองโมเลกุลแบบกระจาย

ได้รับการตรวจสอบและอนุมัติ ให้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
หลักสูตรวิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการคอมพิวเตอร์
เมื่อ วันที่ 22 พฤษภาคม พ.ศ. 2569

อาจารย์ที่ปรึกษา _____
(ผศ.ดร. กษิดิศ ชาญเชี่ยว)

กรรมการสอบโครงงานพิเศษ _____
(ผศ.ดร. ประภาพร รัตนธำรง)

กรรมการสอบโครงงานพิเศษ _____
(ผศ.ดร. ปกป้อง ส่องเมือง)

มหาวิทยาลัยธรรมศาสตร์
คณะวิทยาศาสตร์และเทคโนโลยี

รายงานโครงงานพิเศษ

ของ

นายณัฐนันต์ โพธิ์ประดิษฐ์

เรื่อง

การประยุกต์ใช้ CHECKPOINT/RESTORE ด้วย CRIU สำหรับระบบการประมวลผลแบบจำลองโมเลกุลแบบกระจาย

ได้รับการตรวจสอบและอนุมัติ ให้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
หลักสูตรวิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการคอมพิวเตอร์
เมื่อ วันที่ 22 พฤษภาคม พ.ศ. 2568

อาจารย์ที่ปรึกษา _____
(ผศ.ดร. กษิดิศ ชาญเชี่ยว)

กรรมการสอบโครงงานพิเศษ _____
(ผศ.ดร. ประภาพร รัตนธำรง)

กรรมการสอบโครงงานพิเศษ _____
(ผศ.ดร. ปกป้อง ส่องเมือง)

หัวข้อโครงงานพิเศษ	การประยุกต์ใช้ CHECKPOINT/RESTORE ด้วย CRIU สำหรับระบบการประมวลผลแบบจำลองโมเลกุลแบบกระจาย
ชื่อผู้เขียน	นายณัฐนันต์ โพธิ์ประดิษฐ์
ชื่อปริญญา	วิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการคอมพิวเตอร์
สาขาวิชา/คณะ/มหาวิทยาลัย	สาขาวิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยธรรมศาสตร์
อาจารย์ที่ปรึกษาโครงงานพิเศษ	ผศ.ดร. กษิดิศ ชาญเชี่ยว
ปีการศึกษา	2568

บทคัดย่อ

โครงงานนี้นำเสนอการออกแบบระบบการประมวลผลแบบจำลองโมเลกุลแบบกระจายในลักษณะ (Volunteer Computing) ซึ่งมักมีความไม่แน่นอนสูงจากการที่อาสาสมัครไม่สามารถเปิดคอมพิวเตอร์ได้ตลอดเวลาหรือระบบอาจเกิดการขัดข้องกะทันหัน ผู้วิจัยจึงได้เสนอการประยุกต์ใช้ Checkpoint/Restore ด้วย CRIU สำหรับระบบการประมวลผลแบบจำลองโมเลกุลแบบกระจาย เพื่อสร้างระบบต้นแบบที่สามารถบันทึก ย้ายสถานะ และกู้คืนการประมวลผลได้

ระบบประกอบด้วย Client ที่ใช้โปรแกรม GROMACS ในการประมวลผล CRIU สำหรับการ Checkpoint/Restore และ Middleware ในการควบคุมการทำงานและติดต่อสื่อสารกับ **Server** ในฝั่ง Server มี Coordination Server สำหรับจัดการคิวงานด้วย Apache Artemis และ Storage Servers ที่ประกอบไปด้วย Work Server สำหรับจัดเก็บงานต้นฉบับ, Snapshot Server สำหรับจัดเก็บ Snapshot และ Result Server สำหรับจัดเก็บผลลัพธ์ โดยใช้ MinIO ในการจัดเก็บข้อมูล

กลไกการจัดการความล้มเหลวแบ่งเป็น 2 ลักษณะ คือ การจัดการความล้มเหลวที่จัดการได้ (Graceful Shutdown) ผ่านการดักจับสัญญาณ SIGINT/SIGTERM เพื่อทำ Final Checkpoint และส่งต่อสถานะไปยังเครื่องอื่นทันที และการจัดการความล้มเหลวที่จัดการไม่ได้ ผ่านระบบ Periodic Checkpoint ทุก 2 นาที ซึ่งทำหน้าที่เป็นสัญญาณ Heartbeat หาก Client เงียบหายไป Server จะนำ Snapshot ล่าสุดกลับเข้าสู่คิวงานเพื่อให้เครื่องอื่นรับช่วงต่อได้ทันที

จากการทดสอบประสิทธิภาพด้วยแบบจำลอง 3 ขนาด (ligandCMET, benchMEM และ benchRIB) พบว่าระบบสามารถทำงานตามความต้องการของระบบได้อย่าง

ครบถ้วน โดยแบบจำลองขนาดเล็กและกลางมีประสิทธิภาพดีเยี่ยมในการทำ Checkpoint/Restore แต่แบบจำลองขนาดใหญ่ที่มีจำนวนอะตอมกว่า 2 ล้านอะตอม พบข้อจำกัดด้านเวลาในการบีบอัดและอัปโหลดไฟล์ Snapshot ขนาดใหญ่ ซึ่งกินเวลาเกือบเท่ารอบการบันทึกอัตโนมัติ สรุปได้ว่าเทคนิคนี้สามารถช่วยเพิ่มความทนทานต่อความเสียหาย (Fault Tolerance) ให้กับระบบได้อย่างมีประสิทธิภาพ แต่สำหรับงานขนาดใหญ่มากควรมีการเพิ่มประสิทธิภาพการจัดการความคืบหน้าเพื่อให้คุ้มค่ากับการใช้งานในสภาพแวดล้อมจริง

คำสำคัญ: Checkpointing, การประมวลผลแบบกระจาย, การประมวลผลแบบจำลองโมเลกุล

Thesis Title	Application of Checkpoint/Restore using CRIU for Distributed Molecular Simulation Systems
Author	Mr. Natanan Phopradit
Degree	Bachelor of Science
Major Field/Faculty/University	Computer Science Faculty of Science and Technology Thammasat University
Project Advisor	Asst. Prof. Dr. Kasidit Chanchio
Academic Years	2025

ABSTRACT

This project proposes solution to mitigate processing progress loss in Volunteer Computing systems, which often faces uncertainty of volunteers that typical users will not have their computer turned on all the time, and failure of one's system could happen at any time. The research proposes a design for volunteer distributed computing system of molecular simulation, which utilizes checkpoint/restore function of CRIU in the environment to enable client's capabilities of recovering and migrating processing states at hand to other peers via server to reduce impact to the whole project, promoting high availability and fault tolerance.

The system consists of clients utilizing CRIU for Checkpoint/Restore operations, and a Middleware layer to govern operations and communicate with the servers. On the server side, a Coordination Server manages job queues via Apache Artemis, while Storage Servers - consisting of a Work Server for original tasks, a Snapshot Server for checkpointed states sent from clients, and a Result Server for final outputs - utilize MinIO for data storage.

The fault tolerance mechanism is categorized into two types: Graceful Shutdown, which handles SIGINT/SIGTERM signals by performing a final checkpoint and immediately transfers the snapshot to the server for other clients to carry on; and Non-graceful Failure, handled by periodic checkpoints at regular intervals. These periodic

checkpoints also serve as a heartbeat signal; if a client becomes unresponsive, the server re-enqueues the latest snapshot for another client to resume the work instantly.

Performance testing using three model sizes (ligandCMET, benchMEM, and benchRIB) demonstrated that the system fully meets all functional requirements. While small and medium-sized models showed optimal performance in Checkpoint/Restore tasks, the large-scale model—containing over 2 million atom-revealed limitations in the time required for uploading large snapshot files, which might affect total runtime in system that face high uncertainties. In conclusion, while this technique effectively enhances fault tolerance for distributed computing, further optimization in snapshot management is necessary for massive-scale workloads to be viable in real-world production environments.

Keywords: Checkpointing, Distributed Computing, Molecular Simulation

กิตติกรรมประกาศ

โครงงานพิเศษ 2 นี้ จะไม่อาจสำเร็จลุล่วงไปได้ด้วยดีเลยหากไม่ได้รับการสนับสนุนจากหลายฝ่าย ต้องขอขอบพระคุณ ผู้ช่วยศาสตราจารย์ ดร. กษิดิศ ชาญเชี่ยว อาจารย์ที่ปรึกษา ที่คอยให้คำแนะนำในด้านต่าง ๆ ตั้งแต่ในช่วงค้นหาหัวข้อโครงงานพิเศษ คอยให้แนวคิด ประเด็นต่าง ๆ ที่น่าสนใจ จนภายหลังที่เริ่มดำเนินงาน ถึงแม้ผู้วิจัยจะไม่สามารถทำงานได้ดีอย่างที่อาจารย์อาจคาดหวัง แต่ก็ยังคอยสนับสนุน ให้ความช่วยเหลือในแก้ไขและตรวจสอบความเรียบร้อยของรายงานให้ออกมาสมบูรณ์ คอยตามให้อัปเดตงานเป็นระยะ และเตรียมผู้วิจัยให้พร้อมกับการนำเสนอ

ขอบพระคุณอาจารย์ทั้งในสาขาและนอกสาขาทุกท่าน ที่ให้คำแนะนำตั้งแต่การตัดสินใจในการทำโครงงาน ให้ความรู้ ความช่วยเหลือ มอบโอกาสต่าง ๆ แก่ผู้วิจัยรวมไปถึงเป็นกำลังใจให้ผู้วิจัยมาโดยตลอด

ขอบพระคุณคุณพ่อและคุณแม่ที่เลี้ยงดูเป็นอย่างดี ไม่เคยให้ขาดตกบกพร่องหรือลำบากแต่อย่างไรเลย ตั้งแต่ก่อนศึกษาที่มหาวิทยาลัยจนผู้วิจัยได้จัดทำโครงงานพิเศษนี้ คอยเป็นกำลังใจไม่ว่าจะตอนที่ผู้วิจัยจะมีสุขหรือลำบาก และสนับสนุนผู้วิจัยในทุกการตัดสินใจ

ขอขอบคุณเพื่อน พี่ น้องทุกคนที่คอยเป็นกำลังใจ ทักทายเมื่อเจอหน้ากัน ชวนออกไปทานข้าว พาทำกิจกรรมด้วยกัน เป็นกำลังใจให้กันในวันที่ลำบาก นอกจากนี้ยังสามารถหาเรื่องราวชาวบ้านมาพูดคุยเสมือนตนปราศจากงานและภาระหน้าที่ในชีวิต ทำให้ผู้วิจัยผ่านวันที่ลำบากไปได้ในทุกๆครั้ง และทำให้ผู้วิจัยไม่รู้สึกโดดเดี่ยวเลย

ขอบคุณคณะแพทย์ พยาบาล และบุคลากรทางการแพทย์ทุกท่านจากโรงพยาบาลธรรมศาสตร์เฉลิมพระเกียรติ ที่ให้การรักษาผู้วิจัยเป็นอย่างดีในช่วงที่ผู้วิจัยป่วยขณะทำโครงงานในช่วงเทอมการศึกษา 1 ให้คำแนะนำและบรรเทาอาการป่วยให้เบาบางลง และกลับมาดำเนินชีวิตและทำการศึกษาโครงงานตามปกติได้อย่างรวดเร็ว

สุดท้ายนี้ ขอขอบคุณตัวผู้วิจัยเองที่ตัดสินใจทำโครงงานพิเศษ ฝ่าฟันอุปสรรค์ต่าง ๆ บาลานซ์การทำงานและชีวิตส่วนตัวต่าง ๆ ผู้วิจัยรู้สึกภูมิใจอย่างมาก นอกจากจะได้เป็นผลงานชิ้นหนึ่งแล้ว ผู้วิจัยยังได้ความรู้มากมายจากการศึกษาเครื่องมือและเทคนิควิธีต่างๆ ด้วยตนเอง ผู้วิจัยคาดหวังว่าโครงงานนี้จะเป็นประโยชน์ต่อความก้าวหน้าในเทคโนโลยี และเป็นแรงบันดาลใจให้กับนักศึกษาวิทยาการคอมพิวเตอร์ในการทำโครงงานพิเศษในอนาคต

นายณัฐนันต์ โพธิ์ประดิษฐ์

สารบัญ

หน้า

บทคัดย่อ 6

ABSTRACT 8

กิตติกรรมประกาศ 10

สารบัญ 11

สารบัญภาพ 14

บทที่ 1 บทนำ 1

1.1 ความเป็นมาและความสำคัญของโครงงาน 1

1.2 วัตถุประสงค์ 2

1.3 ขอบเขตของโครงงาน 3

1.4 สมมุติฐานของงานวิจัย 3

1.5 ประโยชน์ของโครงงาน 3

1.6 ข้อจำกัดของโครงงาน 4

บทที่ 2 วรรณกรรมและงานวิจัยที่เกี่ยวข้อง 6

2.1 แนวคิดทฤษฎีที่เกี่ยวข้อง 6

2.1.1 Molecular Dynamics 6

2.1.2 Distributed Computing 6

2.1.3 Volunteer computing 7

2.1.4 Message Broker 8

2.1.5 Checkpointing 9

2.2 งานวิจัยที่เกี่ยวข้อง 9

2.2.1 BOINC: A Platform for Volunteer Computing 9

2.2.2 Run-time Application Migration using Checkpoint/Restore In Userspace 10
บทที่ 3 วิธีการวิจัย 12
3.1 ภาพรวมของโครงงาน 12
3.1.1 Client 13
3.1.2 Server 14
3.2 การวิเคราะห์ขอบเขตและความต้องการของระบบ 17
3.2.1 ขอบเขตของระบบ 17
3.2.2 ความต้องการของระบบ 18
3.2.3 ข้อกำหนดและข้อจำกัดของระบบ 20
3.3 การดำเนินงาน 22
3.3.1 การจัดเตรียมฮาร์ดแวร์และซอฟต์แวร์ 22
3.3.2 ขั้นตอนการทำงาน 24
บทที่ 4 ผลการดำเนินงาน 30
4.1 ผลการประเมินประสิทธิภาพ 30
4.2 บทสรุปและการวิเคราะห์ผลการดำเนินงาน 31
บทที่ 5 สรุป 36
5.1 ข้อเสนอแนะ 37
ภาคผนวก 41
ภาคผนวก ก. ภาพโครงสร้างของระบบตัวอย่าง 42
ภาคผนวก ช. Use Case Diagram ของระบบ 43
ภาคผนวก ค. Network Topology ของระบบภายใน OpenStack 44
ภาคผนวก ง. ภาพการทำงานของระบบระหว่างแต่ละ Component 45

สารบัญตาราง

หน้า

ตารางที่ 4-1 การประเมินประสิทธิภาพของการประมวลผลแบบจำลองโมเลกุล 30

ตารางที่ 4-2 การประเมินประสิทธิภาพของการบันทึกความคืบหน้าการประมวลผลแบบจำลองโมเลกุล 31

สารบัญภาพ

หน้า

ภาพที่ 2.1-1 การสื่อสารระหว่าง Client และ Server ของ BOINC 7

ภาพที่ 3.1-1 ภาพโครงสร้างของระบบตัวอย่าง 12

ภาพที่ 3.1-2 ส่วนประกอบภายใน Client 14

ภาพที่ 3.1-3 ส่วนประกอบภายใน Server 15

ภาพที่ 3.2-1 Use Case Diagram ของระบบ 17

ภาพที่ 3.3-1 Network Topology ของระบบภายใน OpenStack 22

ภาพที่ 3.3-2 แผนผังขั้นตอนการจัดการคำของานระหว่าง Client และ Server 25

ภาพที่ 3.3-3 แผนผังการรันการประมวลผลตามประเภทงานของ Client 26

ภาพที่ 3.3-4 แผนผังการบันทึกความคืบหน้าอัตโนมัติภายใน Middleware ของ Client 27

ภาพที่ 3.3-5 แผนผังการบันทึกความคืบหน้าอัตโนมัติภายใน Middleware ของ Server 27

ภาพที่ 3.3-6 แผนภาพการจัดการความล้มเหลวของ Client และการบันทึกไปยัง Server 29

ภาพที่ 3.3-7 ภาพการทำงานของระบบระหว่าง Client และ Server 29

ภาพที่ 4.2-1 แผนภูมิแท่งแสดงขนาดของไฟล์งาน (.tpr) และขนาดของผลลัพธ์ 32

ภาพที่ 4.2-2 แผนภูมิเส้นแสดงขนาดของไฟล์งานและประสิทธิภาพการประมวลผล 32

ภาพที่ 4.2-3 แผนภูมิเส้นแสดงขนาดของผลลัพธ์และระยะเวลาอัปโหลดผลลัพธ์ 33

ภาพที่ 4.2-4 แผนภูมิแท่งแสดงระยะเวลาเฉลี่ยในแต่ละกระบวนการบันทึกความคืบหน้า (Checkpoint, Compression และ Upload) ของการประมวลผลไฟล์ ligandCMET benchMEM และ benchRIB 34

ภาพที่ 4.2-5 แผนภูมิเส้นแสดงขนาดของ Snapshot และระยะเวลาการ Restore 35

บทที่ 1
บทนำ

1.1 ความเป็นมาและความสำคัญของโครงงาน

ในปัจจุบัน การประมวลผลที่มีประสิทธิภาพสูงมีความจำเป็นอย่างยิ่งต่อความก้าวหน้าทางวิทยาศาสตร์ หนึ่งในแนวทางที่เป็นมาตรฐานคือการใช้ระบบการประมวลผลแบบกระจาย (Distributed systems) ซึ่งเป็นการทำงานร่วมกันของคอมพิวเตอร์จำนวนมากผ่านระบบเครือข่ายเพื่อประมวลผลงานโปรเจคขนาดใหญ่ร่วมกัน โดยอาศัยการแบ่งงานออกเป็นหน่วยย่อยแล้วกระจายไปยังแต่ละโหนดในระบบก่อนจะนำผลลัพธ์มารวมกัน แนวคิดนี้ได้ขยายตัวไปสู่รูปแบบการประมวลผลโดยอาสาสมัคร (Volunteer computing) ซึ่งเปิดโอกาสให้ผู้ใช้งานทั่วไปสามารถแบ่งปันทรัพยากรคอมพิวเตอร์ที่ไม่ได้ใช้งานเพื่อสนับสนุนงานวิจัยระดับโลกได้

ตัวอย่างของโปรเจคขนาดใหญ่หนึ่ง คือการประมวลแบบจำลองโมเลกุล ซึ่งเป็นเครื่องมือสำคัญที่แก้ไขข้อจำกัดของการศึกษาทางวิทยาศาสตร์ในปัจจุบัน เนื่องจากการทดลองเพื่อศึกษาพฤติกรรมของโมเลกุลในสภาวะจริงมักเผชิญกับข้อจำกัดทั้งทางด้านกายภาพของอุปกรณ์และความละเอียดในการสังเกตการณ์ของมนุษย์ การสร้างสภาพแวดล้อมจำลองเสมือนที่อ้างอิงตามกฎการเคลื่อนที่ของนิวตันเพื่อคาดการณ์ปฏิสัมพันธ์ของสสารในระดับอะตอมจึงช่วยให้สามารถศึกษาปรากฏการณ์ที่อาจทำได้ยากหรือเป็นไปไม่ได้เลยในห้องปฏิบัติการจริง นอกจากนี้ยังประหยัดเวลาค่าใช้จ่าย ลดความผิดพลาดของมนุษย์ได้อย่างมาก สามารถทำซ้ำได้ และปรับสภาพแวดล้อมตั้งต้นได้อย่างสะดวก

อย่างไรก็ตาม การจำลองพฤติกรรมเหล่านี้มีความซับซ้อนอย่างมาก เนื่องจากระบบต้องคำนวณพารามิเตอร์จำนวนมหาศาลที่มีความสัมพันธ์กันอย่างต่อเนื่อง ทำให้ต้องใช้ทรัพยากรประมวลผลและเวลาสูงมาก จนมักเกินขีดความสามารถของคอมพิวเตอร์ทั่วไป ด้วยเหตุนี้ แนวคิดการประยุกต์ใช้การประมวลผลโดยอาสาสมัคร (Volunteer Computing) กับการประมวลผลแบบจำลองโมเลกุลจึงเกิดขึ้น ตัวอย่างที่ประสบความสำเร็จในปัจจุบันคือโปรเจกต์ Folding@home ที่ให้อาสาสมัครร่วมกันประมวลผลการพับตัวของโปรตีน ช่วยให้นักวิจัยเข้าใจพฤติกรรมของโมเลกุลที่ซับซ้อนและค้นหาแนวทางการรักษาโรคต่าง ๆ ได้รวดเร็วและประหยัดค่าใช้จ่ายได้มาก

แต่จากการศึกษา พบว่าระยะเวลาที่ผู้คนใช้งานคอมพิวเตอร์ต่อวันมีค่าเฉลี่ยอยู่ที่ 2 ชั่วโมง 52 นาที ซึ่งเป็นเพียงร้อยละ 11.94 ต่อวัน [1] ดังนั้น หากงานตกค้างอยู่ที่ Client และไม่ได้รับการประมวลผล อาจทำให้ต้องเสียโอกาสการประมวลผลไปอย่างเปล่าประโยชน์หากยังมีอาสาสมัครคนอื่นที่ยังสะดวกในการทำงาน นอกจากนี้ ยังหลีกเลี่ยงไม่ได้ว่าคอมพิวเตอร์อาจเกิดข้อผิดพลาดระหว่างการประมวลผล ซึ่งอาจทำให้สูญเสียความคืบหน้าของการประมวลผล หากแต่มีวิธีที่สามารถย้ายการทำงานให้อาสาสมัครคนอื่นเพื่อประมวลผลต่อไปได้ และการออกแบบระบบให้มีความสามารถ Fault tolerance

ดังนั้น โครงงานนี้จึงมุ่งเน้นการออกแบบและพัฒนาระบบการประมวลผลแบบจำลองโมเลกุลแบบกระจายที่ทนทานต่อความล้มเหลวในระดับ Client อย่างมีประสิทธิภาพ โดยการประยุกต์ใช้กลไก Checkpoint/Restore ด้วย CRIU เพื่อจัดการสถานะการประมวลผลในสองลักษณะหลัก ลักษณะแรก คือการจัดการความล้มเหลวที่จัดการได้ (Graceful Shutdown) ซึ่ง Client จะทำการบันทึก Snapshot และย้ายสถานะการทำงานไปยัง Server ทันทีเพื่อส่งต่อภาระงานไปยังคอมพิวเตอร์เครื่องอื่นในระบบ และลักษณะสอง คือการจัดการความล้มเหลวที่จัดการไม่ได้ผ่านระบบ Periodic Checkpoint ที่จะบันทึกความคืบหน้าโดยอัตโนมัติเป็นระยะและทำหน้าที่เป็นสัญญาณ Heartbeat ของ Client ในตัว เมื่อ Client เกิดความล้มเหลวที่จัดการไม่ได้ เช่น System Failure หรือไฟดับ จะทำการอนุญาตให้ Client เดิมหรือเครื่องอื่นในระบบสามารถกู้คืนการประมวลผลจากจุดล่าสุดได้ทันที ช่วยลดการสูญเสียทรัพยากรจากการคำนวณซ้ำและเพิ่มประสิทธิภาพการใช้ทรัพยากรสูงสุดภายใต้สภาพแวดล้อมการประมวลผลที่มีความไม่แน่นอนสูง

1.2 วัตถุประสงค์

- เพื่อศึกษาหลักการออกแบบเบื้องต้นของระบบการประมวลผลแบบกระจายในลักษณะ Volunteer computing
- เพื่อศึกษาและวิเคราะห์กลไกการทำงานของการประมวลผลแบบจำลองโมเลกุลบนสภาพแวดล้อมการประมวลผลแบบกระจาย
- เพื่อออกแบบและสร้างระบบต้นแบบสำหรับการประมวลผลแบบจำลองโมเลกุลแบบกระจายในรูปแบบ Volunteer computing
- เพื่อศึกษาการใช้งาน Checkpoint/Restore ในการ Migrate process ระหว่างคอมพิวเตอร์ในระบบ และ Recover ความคืบหน้าของงานต่อจาก Snapshot เดิมเพื่อป้องกันการสูญเสียความคืบหน้า

1.3 ขอบเขตของโครงงาน

- ระบบ Volunteer computing ในโครงงานนี้จะเป็นขนาดตัวอย่าง โดยระบบขั้นต่ำจะมี Client ที่สามารถรับงานจาก Server มาประมวลผลและส่งผลลัพธ์กลับไปได้ และ Server สามารถจัดการงานให้ Client สามารถนำไปทำได้
- สำหรับ Middleware ภายใน Client จะใช้ GROMACS ในการประมวลผลแบบจำลองโมเลกุล และ CRIU ในการทำ Checkpoint/Restore
- Client รองรับไฟล์โครงสร้างระบบ (.tpr) สามารถดำเนินการประมวลผลจำลองโมเลกุลได้ และไฟล์สถานะความคืบหน้าในรูปแบบ CRIU Image (.tar.gz) สามารถคืนค่าความคืบหน้าการประมวลผลได้
- ครอบคลุมการจัดการความล้มเหลวที่จัดการได้ (Graceful Shutdown ผ่านสัญญาณ SIGINT/SIGTERM) และความล้มเหลวที่จัดการไม่ได้ (Unplanned Failure ผ่านสัญญาณ SIGKILL) โดยทั้งสองกรณี ระบบสามารถจัดการการย้ายสถานะการประมวลผลไปยัง Client เครื่องอื่นหรือ Client เดิมที่พร้อมทำงานได้

1.4 สมมุติฐานของงานวิจัย

- การกำหนดระดับความสำคัญ (Priority) ให้กับงานประเภท Snapshot สูงกว่างานต้นฉบับ จะช่วยให้ระบบเลือกประมวลผลความคืบหน้าของงานที่ค้างอยู่ต่อจากเดิมก่อนการเริ่มงานใหม่
- การทำ Periodic Checkpoint ในรอบระยะเวลาที่เหมาะสม (2 นาที) กับการใช้กลไก Last Value Queue (LVQ) เป็นสัญญาณ Heartbeat ว่า Client ยังคงทำงานอยู่ หาก Client ไม่ส่ง Snapshot ภายในระยะเวลาหนึ่งจะถือว่า Client นั้นเกิดข้อผิดพลาด และเปิดให้ Client อื่นสามารถนำความคืบหน้าการประมวลผลไปทำต่อ วิธีนี้จะช่วยป้องกันการสูญเสียข้อมูลความคืบหน้าในกรณีที่ Client เกิดความล้มเหลวฉับพลันได้
- กลไกการดักจับสัญญาณ (Signal Handler) จะช่วยให้ Client สามารถทำ Final Checkpoint และ Migrate งานกลับสู่คิวหลักได้ทันทีเมื่อผู้ใช้งานสั่งปิดโปรแกรมปกติ

1.5 ประโยชน์ของโครงงาน

- ลดการสูญเสียทรัพยากรการประมวลผล ช่วยให้งานวิจัยไม่หยุดชะงักเมื่ออาสาสมัครหยุดใช้งานคอมพิวเตอร์ โดยการย้ายงานที่ค้างอยู่ไปให้ผู้อื่นทำต่อได้ทันที

- เพิ่มความทนทานต่อความล้มเหลว จัดการ Fault Tolerance สำหรับระบบ Volunteer Computing ที่มีความไม่แน่นอนสูง
- ได้แนวทางและ Middleware ต้นแบบที่สามารถนำไปปรับใช้กับซอฟต์แวร์ประมวลผลทางวิทยาศาสตร์ประเภทอื่นได้
- สามารถนำระบบต้นแบบไปต่อยอดในการปรับปรุงการจัดการบริหารงาน หรือจัดการความล้มเหลวของการประมวลผลแบบกระจายอื่นได้

1.6 ข้อจำกัดของโครงงาน

- ระบบถูกออกแบบและทดสอบภายในสภาพแวดล้อมจำลอง (Virtual Environment) บน OpenStack โดยใช้ Virtual Machine ที่ทำงานบนระบบปฏิบัติการ Ubuntu เป็นหลัก การศึกษานี้จำกัดจำนวนเครื่อง Client ไว้ที่ 3 เครื่อง เพื่อเป็นเพียงระบบขนาดตัวอย่างในการทดสอบกลไกพื้นฐานเท่านั้น
- ขอบเขตของระบบจำกัดอยู่เพียงการใช้ GROMACS สำหรับการประมวลผล และ CRIU สำหรับการทำ Checkpoint/Restore ซึ่งอาจไม่ครอบคลุมซอฟต์แวร์หรือเทคนิคการประมวลผลอื่น ๆ งานที่ระบบรองรับจำกัดเฉพาะไฟล์โครงสร้างระบบ (.tpr) หรือไฟล์ Snapshot ที่เกิดจาก GROMACS และ CRIU เท่านั้น
- ความสามารถในการทำ Checkpoint ขึ้นอยู่กับการที่โปรแกรมสามารถดักจับ (Catch) สัญญาณได้ ทำให้ไม่สามารถจัดการความล้มเหลวบางประเภทได้ทันที เช่น Kernel Failure, Hardware Failure, Filesystem Crashes หรือสัญญาณ SIGKILL ที่หยุดการทำงานของโพรเซสโดยไม่สามารถตรวจจับได้ ซึ่งจะใช้ประโยชน์จากการบันทึกความคืบหน้าโดย Periodic Checkpoint
- การวิจัยนี้ไม่ได้รวมถึงผลกระทบจากการที่ Client เกิดอาการค้าง (Freeze) หรือความผิดพลาดจากการทำงานล่าช้า (Delay) ของ Client เอง
- เนื่องจากงานวิจัยให้ความสนใจในการป้องกันความล้มเหลวภายใน Client เป็นหลัก ดังนั้นจึงกำหนดให้ Server จะไม่เกิดความล้มเหลวในระหว่างการทำงานของระบบ
- ระบบกำหนดให้อินเทอร์เน็ตมีความเสถียร โดยไม่ได้พิจารณาผลกระทบจากความหน่วงของเครือข่าย (Network Latency) หรือกรณีเครือข่ายตัดขาด (Network Disconnection) ระหว่างการทำงาน
- Client แต่ละเครื่องถูกออกแบบให้สื่อสารผ่าน Public Internet ไปยัง Server เท่านั้น และไม่สามารถติดต่อสื่อสารระหว่าง Client กันเองได้

- ในกรณีที่ไฟล์โครงสร้างมีขนาดใหญ่มากและเกิดความล้มเหลวบ่อยครั้ง การใช้ระบบบันทึกความคืบหน้านี้อาจไม่คุ้มค่าเมื่อเทียบกับทรัพยากรที่เสียไปในการจัดการไฟล์ Snapshot
- ในกรณีที่ Snapshot จากการบันทึกอัตโนมัติส่งล่าช้า จนทำให้ Server Promote งานเข้าคิวหลักและเครื่องอื่นรับงานไปทำซ้ำ ระบบจะใช้วิธีการปฏิเสธ (Reject) ผลลัพธ์ที่ซ้ำซ้อนจากฝั่ง Server เท่านั้น แต่ไม่สามารถป้องกันการทำงานซ้ำซ้อนที่เกิดขึ้นไปแล้วได้
- ระบบจะให้ความสำคัญและจัดเก็บเฉพาะ Snapshot ล่าสุดของแต่ละงานเท่านั้น โดยข้อมูลเดิมที่ค้างอยู่ใน Snapshot Server จะถูกแทนที่ด้วยข้อมูลใหม่เสมอ

บทที่ 2
วรรณกรรมและงานวิจัยที่เกี่ยวข้อง

2.1 แนวคิดทฤษฎีที่เกี่ยวข้อง

2.1.1 Molecular Dynamics

Molecular dynamics คือการใช้วิธีการทางคอมพิวเตอร์ในการจำลองการเคลื่อนที่ของสะสารในหนึ่งหน่วยเวลาตามกฎการเคลื่อนที่ของนิวตัน ระบบถูกควบคุมด้วยข้อกำหนดในทางฟิสิกส์ในสภาพแวดล้อมที่ถูกกำหนดโดยผู้ทดลอง ซึ่งตัวแปรทุกอย่างถูกคำนวณซ้ำเรื่อย ๆ จนระบบเข้าสู่สมดุลในระยะเวลาที่กำหนด ทำให้เห็นวิวัฒนาการของระบบจากการปฏิสัมพันธ์ผ่านฟังก์ชันพลังงานศักดิ์ของระบบ [2] ซึ่งการศึกษาด้วย Molecular dynamics ทำให้เข้าใจความสัมพันธ์ของโมเลกุลที่ซับซ้อนในระดับ Femtosecond ที่ทำการทดลองจริงได้ยาก และการคำนวณด้วยหลักทางสถิติไม่สามารถให้ผลลัพธ์ที่แม่นยำได้ นอกจากนี้ยังช่วยลดความผิดพลาดของมนุษย์ในการทดลองและสิ่งรบกวนระบบที่หลีกเลี่ยงได้ยากที่อาจทำให้การวิเคราะห์คลาดเคลื่อนได้

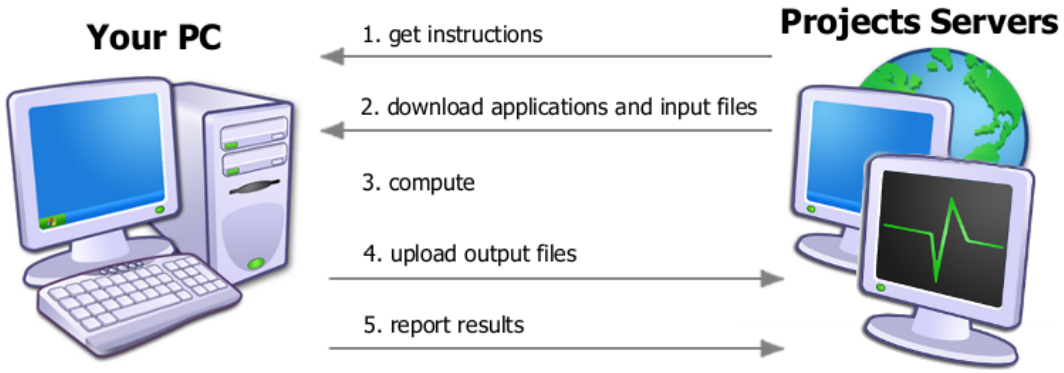
2.1.2 Distributed Computing

Distributed computing หรือการประมวลผลแบบกระจาย คือการประมวลผลคอมพิวเตอร์ด้วยระบบที่ประกอบด้วยทรัพยากรการประมวลผลหรือโหนดจำนวนหนึ่งที่ทำงานอย่างเป็นอิสระต่อกัน ไม่ว่าจะร่วมอยู่กันในสถานที่หนึ่งหรือกระจายอยู่ในสถานที่อื่น ทำงานร่วมกันเพื่อตอบสนองเป้าหมายเดียวกัน มีการสื่อสารกันผ่านตัวกลางเพื่อแลกเปลี่ยนข้อมูลและทำงานร่วมกัน และมีการจัดการสมาชิกภายในเครือข่าย โดยที่ผู้ใช้งานจะเห็นเป็นระบบอันหนึ่งอันเดียวกัน ไม่สังเกตไม่ได้ว่าโพรเซสหรือข้อมูลและระบบการจัดการถูกกระจายไปยังโหนดต่างๆในเน็ตเวิร์ค [3]

[3] ได้เสนอว่า เพื่อให้การสร้างระบบการประมวลผลแบบกระจายนั้นมีประสิทธิภาพและประโยชน์สูงสุด ระบบควรถูกออกแบบโดยที่เข้าถึงทรัพยากรได้ง่าย ระบบควรบดบังการทำงานเบื้องหลัง ไม่ว่าจะเป็นการทำงานด้วยกันของหลายโหนด การจัดการทรัพยากร หรือแม้กระทั่งความล้มเหลวของระบบ ระบบควรเปิดให้ใช้งานหรือนำไปใช้ร่วมกับระบบอื่นได้ง่าย และระบบควรขยายได้

2.1.3 Volunteer computing

Volunteer computing คือระบบการประมวลผลแบบกระจายรูปแบบหนึ่ง ที่ผู้ใช้งานคอมพิวเตอร์จากสถานที่ที่ต่างกันสามารถแบ่งปันทรัพยากรการประมวลผล (หน่วยประมวลผล พื้นที่เก็บข้อมูล อินเตอร์เน็ต) ในการร่วมทำโปรเจควิจัยร่วมกัน โดยการแบ่งงานในโปรเจคเป็นงานย่อย ๆ จากนั้นจึงกระจายการประมวลผลไปยังคอมพิวเตอร์ต่าง ๆ เพื่อให้สามารถทำงานได้พร้อมกัน โดยเมื่อรวมหน่วยประมวลเข้าด้วยกัน ไม่ว่าจะเป็นคอมพิวเตอร์ โทรศัพท์ โน้ตบุ๊ก สามารถให้ศักยภาพการคำนวณที่เทียบเท่ากับหนึ่งซุปเปอร์คอมพิวเตอร์ได้ โดยทั่วไปทรัพยากรการประมวลผลคอมพิวเตอร์ถูกใช้งานจริงเพียง 10-15% [4] ดังนั้นทรัพยากรที่เหลือจึงมีศักยภาพมากเมื่อสามารถที่จะนำมารวมกันเพื่อประมวลผลในงานขนาดใหญ่ที่อาจใช้เวลาเป็นพัน ๆ ปีหากทำบนคอมพิวเตอร์เครื่องเดียว



ภาพที่ 2.1-1 การสื่อสารระหว่าง Client และ Server ของ BOINC

จากตัวอย่างระบบการประมวลผลแบบกระจาย BOINC [5] จากภาพที่ 2-1 จะเป็นรูปแบบการสื่อสารระหว่าง Client และ Server โดยจะมีขั้นตอนการทำงานเป็น

1. คอมพิวเตอร์เลือกงานจาก Schedule Server ซึ่งจะคำนึงถึงทรัพยากรที่คอมพิวเตอร์มี
2. คอมพิวเตอร์ดาวน์โหลดโปรแกรมประมวลผลโปรเจคและไฟล์อินพุตจาก Data Server ในการทำงานของโปรเจค
3. คอมพิวเตอร์รันโปรแกรมและทำการประมวลผล ได้เป็นผลลัพธ์การทำงาน
4. คอมพิวเตอร์อัปโหลดผลลัพธ์ไปยัง Data Server
5. คอมพิวเตอร์รายงานความสำเร็จ และรับงานใหม่จาก Schedule Server

ในปัจจุบันมีโปรเจค Volunteer computing มากกว่า 30 โปรเจคในศาสตร์แขนงต่าง ๆ มีคอมพิวเตอร์รวมกันกว่า 900,000 เครื่อง รวมเป็น CPU กว่า 5 ล้านหน่วยและ GPU อีกกว่า

560,000 หน่วย ซึ่งให้การประมวลผลรวมกว่า 100 Petaflops [6] ทั้งหมดนี้ด้วยความร่วมมืออย่างสมัครใจของผู้ใช้ทั่วโลก ดังนั้น Volunteer computing จึงเป็นส่วนสำคัญที่จะสามารถสร้างการเปลี่ยนแปลงในแวดวงวิทยาศาสตร์อย่างมากที่การประมวลผลแบบเดิมไม่อาจทำได้

2.1.4 Message Broker

Message Broker [7] คือซอฟต์แวร์ที่ทำหน้าที่ตัวกลางของการสื่อสาร เพื่อจัดการการรับส่งข้อมูลระหว่างแอปพลิเคชันหรือเซอร์วิสที่แตกต่างกัน โดยมีจุดประสงค์หลักเพื่อการแยกความสัมพันธ์ระหว่างผู้ส่ง (Producer) และผู้รับ (Consumer) ลดการพึ่งพาการทำงานลง (Tight Coupling) Message Broker จะทำหน้าที่รับข้อความ นำมาจัดเก็บในคิว (Queue) หรือหัวข้อ (Topic) และดำเนินการส่งต่อตามที่กำหนดไว้ ซึ่งกระบวนการนี้ช่วยให้ระบบสามารถรองรับโปรโตคอลที่ต่างกัน หรือการประมวลผลที่ใช้เวลาไม่เท่ากันได้ ทำให้ระบบโดยรวมมีความยืดหยุ่นและ Scale ได้ดีกว่าการเชื่อมต่อแบบ HTTP Request ทั่วไป

2.1.4.1 Message broker models

1) Point-to-Point Messaging Model

Point-to-Point หรือ Message Queueing ข้อมูลจะถูกจัดเก็บในลักษณะคิวลำดับก่อนหลัง (FIFO - First In, First Out) โดยมีหลักการสำคัญคือ หนึ่งข้อความต่อหนึ่งผู้รับ เมื่อผู้ส่งส่งข้อความเข้าสู่คิว ข้อความนั้นจะถูกเก็บไว้จนกว่าจะมีผู้รับมาดึงไปประมวลผล และเมื่อประมวลผลเสร็จสิ้น (ได้รับการ Acknowledge) ข้อความจะถูกลบทิ้งจากคิวทันที ตัวอย่างระบบที่ใช้งาน เช่น ระบบการตัดยอดเงินในธนาคาร หรือระบบการสั่งซื้อสินค้าที่ต้องประมวลผลตามลำดับการจอง เพื่อป้องกันปัญหา Race Condition ในระบบที่กระจายตัว

2) รูปแบบการกระจายข้อมูลแบบ Pub/Sub (Publish/Subscribe Model)

Publish/Subscribe หรือ Pub/Sub เน้นการกระจายข้อมูลจากแหล่งเดียวไปยังผู้รับหลายแหล่งพร้อมกัน (One-to-Many หรือ Many-to-Many) โดยใช้แนวคิดของ Topic เป็นหัวข้อคัดแยกประเภทข้อมูล ผู้ส่ง (Publisher) จะทำหน้าที่เพียงส่งข้อมูล (Publish) ไปยัง Topic ที่เกี่ยวข้อง โดยไม่จำเป็นต้องทราบว่ามีใครรอรับข้อมูลอยู่บ้าง ในขณะที่ผู้รับ (Subscriber) จะลงทะเบียน (Subscribe) เพื่อติดตาม Topic ที่ตนเองสนใจ เมื่อมีข้อมูลใหม่เข้ามา Message Broker จะทำการบรอดแคสต์ หรือส่งข้อความไปยังทุกปลายทางที่ Subscribe ไว้ โมเดลนี้จึงสำคัญต่อระบบการแจ้งเตือน (Notification) และการอัปเดตข้อมูลแบบ Real-time ในระบบขนาดใหญ่

2.1.5 Checkpointing

Checkpointing หรือการบันทึกสถานะ ในความหมายของระบบคอมพิวเตอร์ คือ การบันทึกสถานะของระบบหรือส่วนหนึ่งของระบบ ไม่ว่าจะเป็นโพรเซส เมมโมรี หรือคอมพิวเตอร์จำลอง (Virtual machine) โดยมีลักษณะการบันทึกเป็น Snapshot ที่รวม Context ของสิ่งที่ต้องการบันทึก ณ เวลาการบันทึก เช่น ข้อมูลในเมมโมรี Page table, CPU register, Network configuration หรือข้อมูลอื่น ๆ ที่สำคัญต่อการคืนค่าการทำงาน และในภายหลังสามารถนำ Snapshot มาคืนค่าการทำงาน เพื่อให้สิ่งที่บันทึกสามารถกลับมาทำงานต่อจากเดิมได้ การทำ Checkpointing เป็นเทคนิคที่ถูกนำไปประยุกต์ใช้ในการสร้างความทนทานต่อความล้มเหลว หรือแม้กระทั่งการกระจายการทำงานในระบบการประมวลผลแบบกระจายเพื่อให้ใช้ทรัพยากรได้อย่างคุ้มค่าที่สุด

2.2 งานวิจัยที่เกี่ยวข้อง

2.2.1 BOINC: A Platform for Volunteer Computing

BOINC [8] เป็นระบบ Open Source Middleware ที่ได้รับความนิยมสูงสุดในงาน VC (Volunteer Computing) BOINC เปิดโอกาสให้ นักวิทยาศาสตร์สร้างและจัดการ VC Project และอาสาสมัครสามารถร่วมช่วยเหลือในงานโปรเจคได้ โดยการที่อาสาสมัครจะติดตั้งโปรแกรม BOINC Client แล้วเลือกโปรเจคที่ต้องการจะสนับสนุน โปรแกรมไคลเอนจะสนับสนุนทั้ง Desktop และ Mobile BOINC ถูกออกแบบมาให้ ทำการคำนวณอยู่เบื้องหลังภายในเครื่องของอาสาสมัคร โดยจะ รันโปรแกรมและกำหนดความสำคัญให้ต่ำที่สุด จำกัดการใช้ทรัพยากรเพื่อไม่ให้กระทบกับการใช้งานปกติของเครื่องขออาสาสมัคร

2.2.1.1 สถาปัตยกรรมและส่วนประกอบของระบบ

โครงสร้างของ BOINC ประกอบด้วยส่วนประกอบหลักที่สื่อสารกันผ่าน HTTP-based RPC ออกแบบเพื่อให้ Modular และ Scalable:

Volunteer: อาสาสมัคร หรือผู้ที่เป็นเจ้าของอุปกรณ์ที่เข้าร่วมในโปรเจค VC และใช้ในการประมวลผล ซึ่งจะติดตั้งโปรแกรม BOINC Client สำหรับการติดต่อขอรับงานในโปรเจคที่เลือก

- **Project Server:** ประกอบด้วยฐานข้อมูลและกระบวนการต่างๆ เช่น **Scheduler** สำหรับแจกจ่ายงาน, **Feeder** สำหรับจัดการคิวในหน่วยความจำ และ **Validator** สำหรับตรวจสอบความถูกต้องของผลลัพธ์

- **Account Manager:** ระบบช่วยอำนวยความสะดวกให้ผู้ใช้งานสามารถจัดการโครงการหลายๆ โครงการได้จากที่เดียว

2.2.2 Run-time Application Migration using Checkpoint/Restore In Userspace

Tošić [9] ได้ทำการศึกษาเชิงประจักษ์ (Empirical Study) เกี่ยวกับความเป็นไปได้ในการนำ Checkpoint/Restore In Userspace (CRIU) มาใช้สำหรับการย้ายแอปพลิเคชันในขณะช่วง Run-Time ระหว่าง Host โดยมุ่งเน้นไปที่สภาพแวดล้อมแบบ Edge Computing และโครงสร้างบนCloud งานวิจัยได้ทดลองการใช้งาน CRIU ร่วมกับ Docker และวิเคราะห์ผลกระทบของ Memory Usage ของแอปพลิเคชันที่มีต่อขนาดของ Checkpoint, ระยะเวลาที่ใช้ และทรัพยากรระบบ

เมื่อใช้ CRIU เพื่อทำ Checkpoint แอปพลิเคชัน จะมีขั้นตอนดังนี้:

1) **Image Creation** สร้างไฟล์เก็บข้อมูลหน่วยความจำ, ไฟล์ที่เปิดอยู่, และการเชื่อมต่อเครือข่าย
2) **Freezing** หยุดการทำงานของโปรเซสชั่วคราว
3) **Image Dumping** บันทึกสถานะลงไฟล์ Checkpoint, และ
4) **Thawing** ปล่อยให้โปรเซสทำงานต่อ

2.2.2.1 ผลการทดลอง

ผลการวิเคราะห์แสดงให้เห็นว่า หน่วยความจำ (Memory) เป็นปัจจัยหลักที่ส่งผลต่อประสิทธิภาพของ CRIU โดยพบว่าเวลาที่ใช้ในการทำ Checkpoint จะแปรผันตรงกับขนาดการหน่วยความจำของคอนเทนเนอร์ ในขณะที่เวลาในการกู้คืน (Restore) จะต่ำกว่าอย่างมีนัยสำคัญ อย่างไรก็ดี CPU Utilization ไม่ได้แปรผันตามขนาดของหน่วยความจำเสมือนที่จัดสรรในระหว่างขั้นตอน Checkpoint และ Restore ดังนั้น Edge Computing จึงควรพิจารณา Trade-Off ระหว่างความถี่ในการทำ Checkpoint และประสิทธิภาพของแอปพลิเคชัน นอกจากนี้ แม้ว่าการใช้งาน CPU จะดูเหมือนไม่ได้เพิ่มขึ้นตามขนาดหน่วยความจำเสมือนที่จัดสรร แต่มันก็ยังคงเป็นปัจจัยจำกัดในการติดตั้งระบบ Checkpointing บน Edge Device ที่มีทรัพยากรจำกัด

สำหรับงานในอนาคต สามารถสำรวจเทคนิคในการลดภาระหน่วยความจำของคอนเทนเนอร์ เพื่อให้สามารถทำ Checkpoint ได้บ่อยขึ้นโดยไม่ส่งผลกระทบต่อประสิทธิภาพของแอปพลิเคชัน นอกจากนี้ ยังน่าสนใจที่จะศึกษาการใช้ CRIU กับแอปพลิเคชันแบบ **Stateful** ที่มีความ

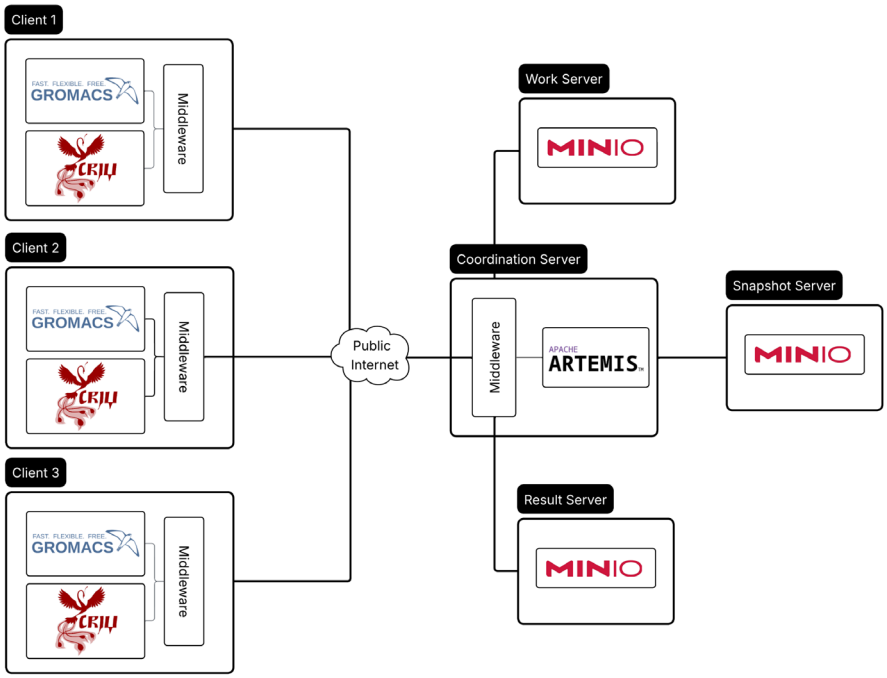
ซับซ้อนมากขึ้น รวมถึงการใช้ CRIU เข้ากับเครื่องมือจัดการคอนเทนเนอร์ (**Orchestration Tools**) อื่นๆ โดยรวมแล้ว การศึกษานี้ช่วยสร้างความเข้าใจในขีดความสามารถและข้อจำกัดของ CRIU ในบริบทของ Edge Computing และให้ข้อมูลเชิงลึกสำหรับการติดตั้งโซลูชัน Checkpointing ในสภาพแวดล้อมที่มีทรัพยากรจำกัด

บทที่ 3
วิธีการวิจัย

3.1 ภาพรวมของโครงงาน

ในการศึกษาการประยุกต์ใช้ Checkpoint/Restore ด้วย CRIU สำหรับระบบการประมวลผลแบบจำลองโมเลกุลแบบกระจาย ผู้วิจัยได้วางแผนการออกแบบสภาพแวดล้อมของระบบการประมวลผลในรูปแบบของ Volunteer distributed system หรือระบบการประมวลผลแบบกระจายโดยอาสาสมัคร ซึ่งเป็นการใช้ Computing resource จากอาสาสมัครที่เข้าร่วมในระบบในการประมวลผลแบบจำลองโมเลกุลร่วมกัน ซึ่งงานจะถูกแบบออกเป็นหน่วยย่อย ๆ จากนั้นจึงกระจายไปยังอาสาสมัครแต่ละคนในการคำนวณผลลัพธ์บนเครื่องคอมพิวเตอร์ของตน เมื่อประมวลผลเสร็จสิ้นจึงส่งกลับไปยัง Server

ในโครงงานนี้ สภาพแวดล้อมการประมวลผลแบบกระจายจะถูกออกแบบอยู่บน OpenStack ซึ่งจะประกอบไปด้วย Virtual Machine (VM) ที่ทำงานร่วมกันในการประมวลผลแบบจำลองโมเลกุลอย่างเป็นอิสระต่อกัน โดยจะกำหนดเป็นอาสาสมัครหรือ Client และระบบควบคุมการประมวลผลหรือ Server



ภาพที่ 3.1-1 ภาพโครงสร้างของระบบตัวอย่าง

3.1.1 Client

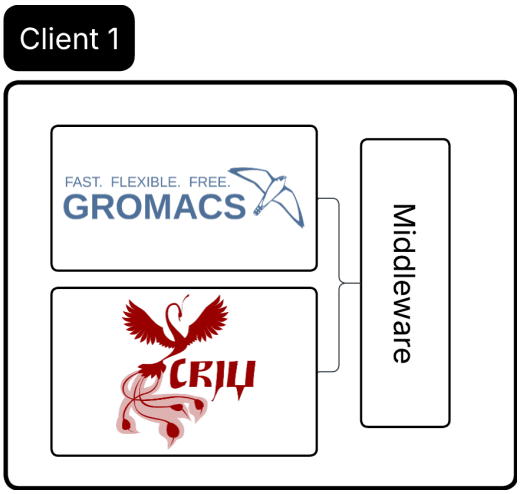
Client เป็นเครื่องคอมพิวเตอร์จำนวนหนึ่งภายในระบบการประมวลผลแบบกระจาย ซึ่งไม่จำเป็นจะต้องอยู่ภายใน Network เดียวกันและการสื่อสารระหว่าง Client และ Server จะสื่อสารกันผ่าน Public Network โดยภายในเครื่อง Client VM จะประกอบไปด้วย

1) GROMACS โปรแกรมสำหรับรับไฟล์งานมารันงานการประมวลผลโมเลกุล
2) CRIU โปรแกรมสำหรับการทำ Checkpoint เพื่อบันทึกสถานะการทำงาน (Snapshot) เมื่อโปรแกรมหยุดการทำงาน และเป็นระยะตามช่วงเวลาที่กำหนด เพื่อป้องกันความล้มเหลวและสามารถกลับมาทำงานต่อได้ และการทำ Restore เพื่อนำ Snapshot กลับมาทำงานต่อ
3) Middleware ถูกเขียนขึ้นด้วยภาษา C++ ทำหน้าที่เป็นตัวกลางในการติดต่อกับ Server ควบคุมการทำงานของ Client ในการทำงานภายในระบบ ไม่ว่าจะเป็นการส่งคำของานไปยัง Server นำงานที่ได้รับจาก Server มารันการประมวลผล (ทั้งในรูปแบบงานต้นฉบับหรือ Snapshot ที่จะนำมา Restore) ดำเนินการ Migrate Snapshot ไปยัง Server เมื่อผู้ใช้ปิดโปรแกรมหรือคอมพิวเตอร์ ควบคุมการทำ Checkpoint เป็นระยะเพื่อป้องกันความล้มเหลว และส่งผลลัพธ์กลับไปยัง Server เมื่อประมวลผลเสร็จสิ้น

การทำงานของ Client หลักจะประกอบไปด้วย

1) การประผลแบบจำลองโมเลกุลแบบกระจาย: Client เมื่อเริ่มการทำงานของโปรแกรม Middleware จะทำการขอรับงานจาก Coordination Server และรับเป็นงานการประมวลผลซึ่งขึ้นอยู่กับว่า Queue จะให้เป็นงานต้นฉบับหรือ Snapshot จากนั้นหากเป็นงานต้นฉบับ จะทำการเรียกโปรแกรม GROMACS และรันงานการประมวลผล แต่หากเป็น Snapshot จะใช้ CRIU ในการ Restore ความคืบหน้าการประมวลผล
2) การบันทึกผลลัพธ์: เมื่อประมวลผลสำเร็จจะทำการส่งผลลัพธ์ไปยัง Server เพื่อจัดเก็บผลลัพธ์ และรับงานการประมวลผลต่อไป
3) การบันทึกความคืบหน้าการประมวลผล ในขณะที่ Client กำลังประมวลผลงาน หาก Client ถูกขัดจังหวะให้ไม่สามารถประมวลผลต่อไปได้ จะมีการดำเนินการดังนี้
    a. หากได้รับสัญญาณที่จัดการได้ (เช่น การปิดเครื่อง, SIGTERM, SIGINT) Middleware จะทำการใช้งาน CRIU เพื่อ Checkpoint การประมวลผลของ GROMACS เป็น Snapshot และทำการส่งไปยัง Coordination Server เพื่อให้จัดเก็บภายใน Snapshot Server และเพิ่มเป็นงานต่อไป

b. เพื่อป้องกันการสูญหายของความคืบหน้า Middleware จะทำการใช้งาน CRIU เพื่อ Checkpoint การประมวลผลของ GROMACS เป็น Snapshot และทำการส่งไปยัง Coordination Server เป็นระยะ ๆ (Periodic Checkpoint) ซึ่งจะถือว่าการส่งนี้เป็นการตรวจสอบ Heartbeat ของ Client ซึ่งหาก Client เกิดความล้มเหลวที่จัดการไม่ได้ (เช่น SIGKILL, Kernel Failure) Periodic Snapshot จะถูก Promote ให้เป็นงานความคืบหน้าและเพิ่มไปยัง Queue เพื่อให้ Client อื่นสามารถรับไปทำต่อได้ แต่ถ้าหาก Client ประมวลผลงานจนสำเร็จ Periodic Snapshot จะถูกกำหนดว่าเป็น Snapshot ที่เสร็จแล้วและ Client จะไม่นำงานไปทำซ้ำ



ภาพที่ 3.1-2 ส่วนประกอบภายใน Client

3.1.2 Server

Server เป็นระบบกลางที่จัดการดูแลโปรเจคการประมวลผลแบบกระจาย ส่วนประกอบหลักของ Server จะประกอบไปด้วย

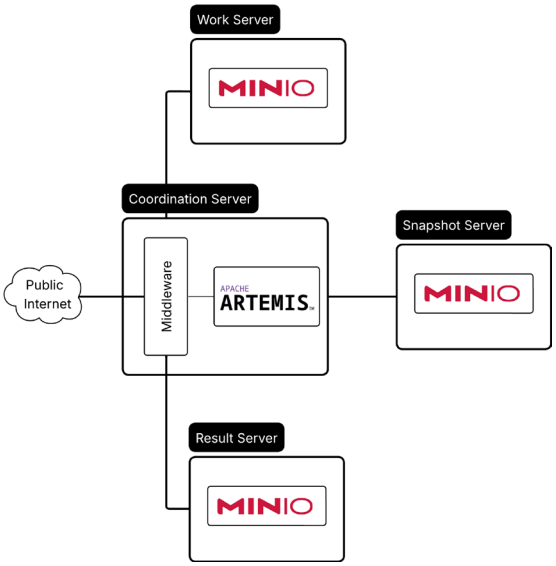
1) Coordination server ทำหน้าที่ติดต่อสื่อสารกับ Client เป็นหลัก รับ Request ไม่ว่าจะเป็นการขอรับงาน การขอส่ง Snapshot หรือการขอส่งผลลัพธ์ ภายใน Coordination Server จะประกอบไปด้วย
   - Apache Artemis โปรแกรมการจัดการ Queue ใช้ในการจัดเรียงงานที่เปิดให้ Client สามารถมาขอรับไปทำได้ ทั้งในรูปแบบของงานต้นฉบับหรือ Snapshot โดยจะ Implement ในรูปแบบ Priority

Queue เพื่อให้งานที่เป็นความคืบหน้าจาก Client ที่เก็บเป็น Snapshot ได้ถูกรับไปก่อน และงานต้นฉบับเป็นลำดับถัดไป

- Middleware ถูกเขียนขึ้นด้วยภาษา C++ ทำหน้าที่ควบคุมการทำงานระหว่างกันของ Client และ Server ทั้งในการเพิ่มงานใน Queue เปิด Queue เพื่อให้ Client สามารถขอรับงาน ประสานไปยัง Storage Servers เพื่อส่งไฟล์ประกอบการทำงานไปยัง Client การรับ Snapshot จาก Client เพื่อจัดเก็บภายใน Snapshot Server หรือการรับผลลัพธ์จาก Client เพื่อจัดเก็บภายใน Result Server

2) Storage Servers จะเป็นหน่วยของ Server ที่ทำหน้าที่ในการจัดเก็บทรัพยากรที่ใช้ในระบบการประมวลผลโมเลกุลแบบกระจาย ภายใน Storage Server แต่ละหน่วยจะทำการติดตั้ง MinIO ในการจัดเก็บทรัพยากรในรูปแบบ Object Storage และ Coordination Server สามารถเข้าถึงได้ด้วย MinIO Client ภายใน Middleware เพื่อส่ง Request มายัง Storage Server ภายในงานวิจัยนี้จะมีหน่วย Storage Servers ดังนี้

- Work Server จัดเก็บงานต้นฉบับและทรัพยากรที่เกี่ยวข้องที่ใช้ในการประมวลผล
- Result server สำหรับจัดเก็บผลลัพธ์การประมวลผล
- Snapshot server เป็นที่รองรับ snapshot ชั่วคราว เพื่อให้ Client สามารถดึงไปทำงานต่อได้



ภาพที่ 3.1-3 ส่วนประกอบภายใน Server

หน้าที่หลักของเซิร์ฟเวอร์จะประกอบไปด้วย

1) การกระจายงานการประมวลผลแบบจำลองโมเลกุล: ภายใน Coordination Server จะใช้ Apache Artemis ในการสร้าง Work Queue เพื่อให้ Client สามารถมาขอรับงานได้ ซึ่งเมื่อได้รับ Request งานจาก Client จะดึง Message งานจาก Queue และส่งทรัพยากรการประมวลผลกลับไปเป็น Response หากเป็นงานต้นฉบับจะทำการขอรับไฟล์งานจาก Work Server ที่ระบุภายใน Message กลับไปยัง Client แต่ถ้าหากเป็น Snapshot จะทำการขอรับ Snapshot จาก Snapshot Server และส่งกลับไปให้ Client ทำการประมวลผล
2) การรวบรวมงานการประมวลผล: เมื่อ Client ทำงานการประมวลผลสำเร็จ จะทำการส่งผลลัพธ์มายัง Coordination Server เพื่อส่งไปจัดเก็บไว้ภายใน Result Server ต่อไป
3) การจัดการ Snapshot จาก Client: เมื่อ Server ได้รับ Request ในการส่ง Snapshot จาก Client จะมีการดำเนินการดังนี้
    - หากเป็น Snapshot ที่มาจากการบันทึกความคืบหน้าเนื่องจาก Client หยุดทำงานปกติ (เช่น การปิดเครื่อง, SIGINT, SIGTERM) จะทำการส่ง Snapshot ไปจัดเก็บภายใน Snapshot Server เพิ่มเป็นงานความคืบหน้าการประมวลผลไว้ภายใน Work Queue และกำหนด Priority ให้สูงกว่างานต้นฉบับ
    - หากเป็น Snapshot ที่มาจากการบันทึกความคืบหน้าเป็นระยะ (Periodic Checkpoint) จะทำการบันทึก Snapshot ไว้ภายใน Snapshot Server และเพิ่ม Message ไว้ภายใน Queue ชั่วคราว หาก Client ไม่ส่งภายในระยะเวลาที่กำหนด จะถือว่า Client เกิดความล้มเหลวแบบที่จัดการไม่ได้ จะทำการ Promote Message ดังกล่าวไปยัง Work Queue เพื่อให้ Client อื่นสามารถนำความคืบหน้าไปทำงานต่อได้

3.2 การวิเคราะห์ขอบเขตและความต้องการของระบบ

3.2.1 ขอบเขตของระบบ

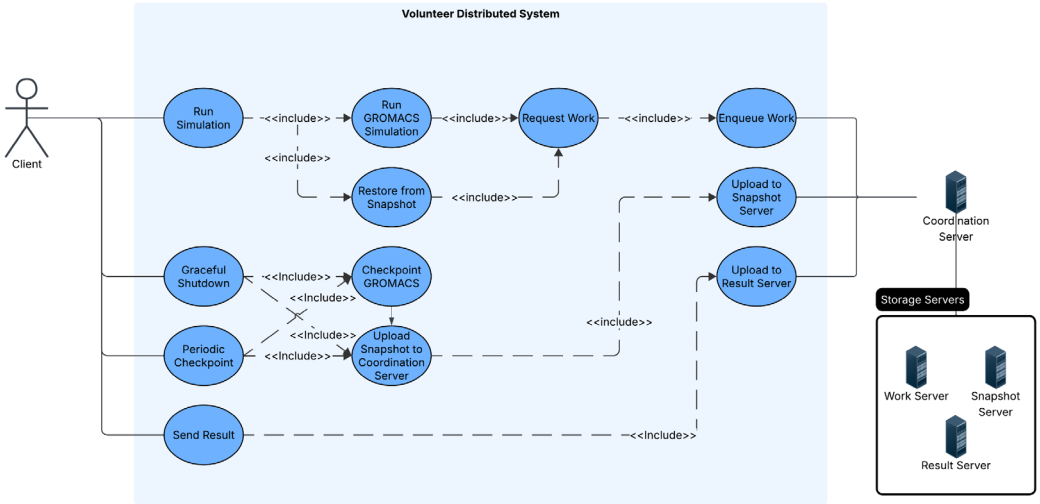
ระบบ Volunteer computing ในโครงงานนี้จะเป็นขนาดตัวอย่าง โดยระบบขั้นต่ำจะมี Client ที่สามารถรับงานจาก Server มาประมวลผลและส่งผลลัพธ์กลับไปได้ และ Server สามารถจัดการงานให้ Client สามารถนำไปทำได้

สำหรับ Middleware ภายใน Client จะใช้ GROMACS ในการประมวลผลแบบจำลองโมเลกุล และ CRIU ในการทำ Checkpoint/Restore

ไฟล์งานที่ใช้ในการศึกษาจะเป็นไฟล์ benchmark จากเว็ปไซต์ Max Planck Institute for Multidisciplinary Sciences [10]

ในการศึกษาจะทำการสุ่มส่งสัญญาณไปยัง Middleware ของ Client เพื่อจำลองการหยุดทำงานของโปรแกรมใน 2 กรณี ได้แก่ความล้มเหลวที่จัดการได้ แทนด้วย SIGINT หรือ SIGTERM และสำหรับความล้มเหลวที่จัดการไม่ได้จะใช้สัญญาณ SIGKILL เพื่อศึกษาการจัดการการบันทึกความคืบหน้าและการส่งต่อความคืบหน้าไปยัง Client อื่นภายในระบบ

ในการวัดประสิทธิภาพ จะเปรียบเทียบระยะเวลาเฉลี่ยที่ใช้ในการประมวลผลเซ็ตของงานการประมวลผลแบบจำลองโมเลกุล ทั้ง 3 ไฟล์ที่มีขนาดที่ต่างกัน เพื่อตรวจสอบระยะเวลาในการทำงานของระบบและความสำเร็จของการทำงานต่าง ๆ ภายในระบบ



ภาพที่ 3.2-1 Use Case Diagram ของระบบ

3.2.2 ความต้องการของระบบ

3.2.2.1 Functional Requirements

1) Servers

1.1) การเพิ่มงานในระบบ: Coordination Server มีการระบุ Metadata (Path, Name, Type) ภายใน Message เพื่อเพิ่มเป็นงานภายใน Work Queue และจัดเก็บไฟล์ลงใน Work server โดยมีการป้องกันชื่อไฟล์ซ้ำ (ภายในระบบใช้ UUID)

1.2) การจัดส่งงาน: Coordination Server มี API เพื่อให้ Middleware ในฝั่ง Client สามารถส่งคำขอ และส่งไฟล์งานให้ Client ตามที่ร้องขอ และมีการระบุประเภทงาน (Topology หรือ Snapshot) ใน Body ของคำขอ

1.3) การรับ Snapshot: Coordination Server มี API เพื่อให้ Middleware ในฝั่ง Client สามารถรับไฟล์ความคืบหน้าการประมวลผลและจัดเก็บภายใน Snapshot Serve

1.3.1) หาก Client ส่ง Snapshot จาก Periodic Checkpoint จะนำส่งไปยัง Snapshot Queue กำหนดให้ข้อความในคิว Snapshot มีคุณสมบัติ Last Value Queue (LVQ) เพื่อให้ Client ได้รับเฉพาะ Snapshot ล่าสุดของแต่ละ Simulation เท่านั้น และกำหนดระยะเวลาที่ Client จะต้องส่ง Snapshot ต่อไปมาอีกครั้ง หากหมดเวลาจะถือว่า Client หยุดการทำงาน และย้ายงานจาก Snapshot Queue ไปยัง Work Queue เพื่อให้ Client สามารถดึงไปดำเนินการต่อได้

1.3.2) หาก Client ส่ง Snapshot จากความล้มเหลวที่จัดการได้ (เช่น การปิดเครื่อง, SIGINT, SIGTERM) จะทำการ Promote งานที่อยู่ภายใน Snapshot Queue ไปยัง Work Queue ทันที

1.4) การจัดเก็บผลลัพธ์: Coordination Server มี API เพื่อให้ Middleware ในฝั่ง Client สามารถส่งผลลัพธ์สุดท้ายหลังจากการทำงาน และนำไปจัดเก็บไว้ภายใน Result Server

1.5) ทำการ mark งานภายใน Snapshot Queue ที่เคยส่งว่าเป็นงานที่สำเร็จแล้ว เพื่อไม่ให้ Client ดึงงานซ้ำ (ทับงานเดิมด้วย LVQ Attribute)

1.6) ป้องกันการส่งไฟล์ผลลัพธ์ซ้ำ: หากได้รับไฟล์ผลลัพธ์เดิม MinIO ภายใน Result Server จะ reject ไฟล์นั้น

2) Clients

2.1) การดึงงาน: Middleware ใน Client เชื่อมต่อกับ Apache Artemis ภายใน Coordination Server เพื่อรับคำสั่งงาน (Dequeue) จาก Work Queue และเมื่อได้รับงาน จะส่ง request เพื่อดาวน์โหลดไฟล์งานจาก Coordination Server

2.2) หากการดึงงานล้มเหลว จะมี Retry และหยุดรอเป็นระยะ (Exponential Backoff)

2.3) การประมวลผลงาน: เมื่อได้รับงานจาก Coordination Server แล้ว Middleware จะเริ่มการประมวลผลทันที

2.4) หากเป็น topology ให้เริ่มการรันใหม่ด้วยโปรแกรม GROMACS เป็น Subprocess

2.5) หากเป็น snapshot จะใช้ CRIU ในการ restore ความคืบหน้าการประมวลผลจาก snapshot ที่เป็น CRIU Image และทำงานต่อจากเดิม

2.6) การสำรองความคืบหน้าอัตโนมัติ: Middleware ใน Clientจะทำ Checkpoint ทุกๆ 2 นาที (หรือตามที่กำหนด) โดยใช้ CRIU และอัปโหลดไปยังยังเซิร์ฟเวอร์ทันที (โปรแกรมการประมวลผลยังคงทำงานต่อ)

2.7) การจัดการความล้มเหลว: เมื่อได้รับสัญญาณ SIGINT หรือ SIGTERM ต้องหยุดการจำลองชั่วคราวเพื่อทำ Final Checkpoint และส่งข้อมูลกลับเซิร์ฟเวอร์ก่อนปิดโปรแกรม

2.8) การส่งผลลัพธ์: เมื่อ GROMACS รันเสร็จสิ้น Middleware จะทำรวบรวมไฟล์ผลลัพธ์บีบอัด และส่งไปยังเซิร์ฟเวอร์ผ่าน API

3.2.2.2 ข้อกำหนดด้านคุณภาพและประสิทธิภาพ (Non-Functional Requirements)

1) ความทนทานต่อความเสียหาย (Fault Tolerance)

1.1) กลไกการเชื่อมต่อใหม่: การดึงไฟล์งานจาก Server Middleware จะมีการ retry ในกรณีที่ไม่สามารถรับไฟล์งานได้ และการส่งผลลัพธ์หรือการส่ง Snapshot ไปยัง Storage Servers หากเกิดการผิดพลาด Coordination Server จะทำการ Retry แบบ Exponential Backoff

1.2) การจัดการความล้มเหลวของ Client: Middleware ภายใน Client จะมีการบันทึก Snapshot เป็นระยะๆ และจะจับสัญญาณ SIGINT/SIGTERM เพื่อทำการสร้าง Checkpoint อัปโหลดขึ้น Server ก่อนปิดตัวลงเพื่อป้องกันการสูญเสียงานที่กำลังประมวลผล

2) ประสิทธิภาพการทำงาน (Performance Optimization)

2.1) การบีบอัดข้อมูล: การโอนย้ายไฟล์ Snapshot อาจมีขนาดใหญ่ Client จะมีการใช้การบีบอัด Snapshot ด้วย pigz (Parallel GZip) เพื่อลดเวลาในการส่งข้อมูลไปยัง Server

3.2.3 ข้อกำหนดและข้อจำกัดของระบบ

เนื่องจากระบบที่ถูกออกแบบภายในงานวิจัย จะเป็นระบบขั้นต่ำและเป็นเพียงขนาดตัวอย่าง นอกจากนี้ระบบถูกออกแบบและทดสอบในสภาพแวดล้อมจำลอง ดังนั้นสำหรับงานวิจัยนี้จะมีข้อกำหนดและข้อจำกัดต่าง ๆ ดังนี้

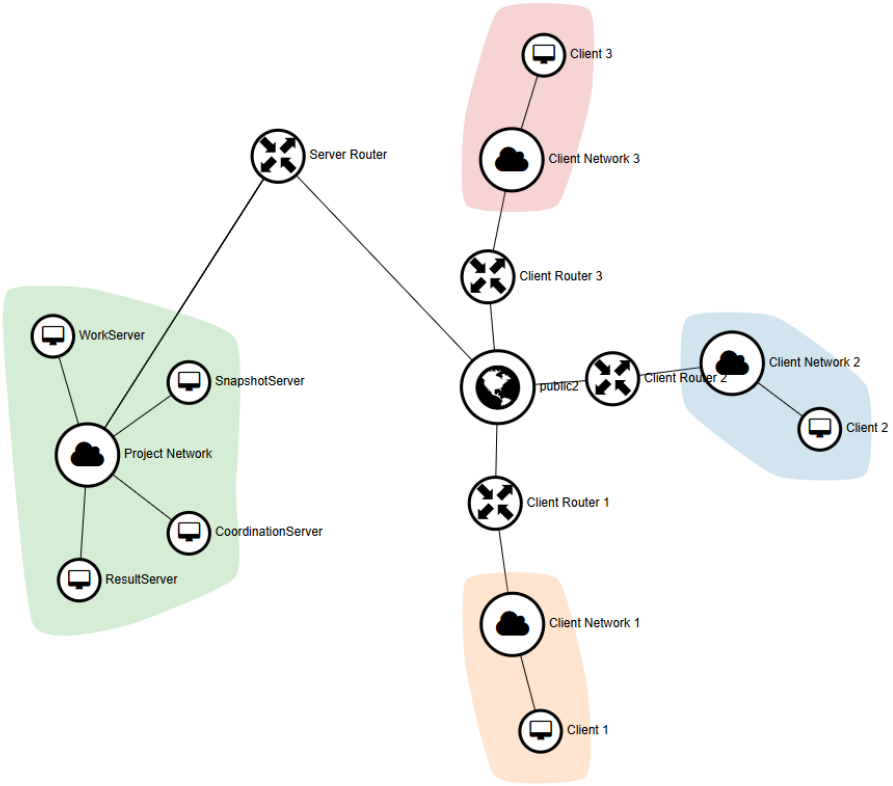
1) ระบบถูกออกแบบในสภาพแวดล้อมจำลอง และระบบ Network จำลอง โดยที่ Client จะติดต่อไปยัง Server ผ่าน Public Internet เท่านั้น และ Client แต่ละเครื่องไม่สามารถติดต่อหากันได้ และจะจำลองว่า Client อยู่ภายในสถานที่ที่ต่างกันเนื่องจากอยู่ภายใน Network ที่ต่างกัน ๆ
2) กำหนดให้ Snapshot มีความสำคัญกว่างานต้นฉบับ ดังนั้น Work Queue จะให้ Priority ของ Snapshot สูงกว่า ซึ่งจะถูกส่งไปยัง Client ก่อนงานต้นฉบับ
3) Server จะไม่ทราบว่าในระบบมี Client ภายในระบบกี่เครื่อง และไม่มีการกำหนดจำนวน Client ที่รองรับได้ สนใจเพียงว่า Client สามารถทำงานได้ครบถ้วน

4) งานวิจัยนี้จะสนใจเพียงความล้มเหลวของ Client ดังนั้นจะกำหนดว่า Servers จะไม่เกิดความล้มเหลวระหว่างที่ยังมี Client ทำงานอยู่ภายในระบบ
5) กำหนดให้ว่าอินเตอร์เน็ตของระบบมีความเสถียร โดยจะไม่พิจารณาผลกระทบจาก Network Latency หรือ Network Disconnection ระหว่างการทำงาน
    a. หากเกิดกรณีที่ทำให้มีการส่ง Snapshot ที่ดีเลย์ ทำให้ Snapshot จาก Periodic Checkpoint นั้นถูก Promote เป็นงานภายใน Work Queue และเกิดการที่ Client อื่นรับความคืบหน้ากลับไปทำทั้งที่ Client เดิมยังทำงานอยู่ (ภายใน Server จะไม่รับผลลัพธ์ที่ซ้ำกัน โดยจะ Reject ผลลัพธ์ที่มีชื่อซ้ำและถูกส่งมาภายหลัง)
6) งานที่ได้รับจาก Server จะกำหนดให้เป็นไฟล์โครงสร้างระบบ พารามิเตอร์ ตำแหน่งและความเร็วของโมเลกุล (System topology, parameters, coordinates and velocities: .tpr) หรือความคืบหน้าการประมวลผล (Snapshot) จาก GROMACS ที่ถูก Checkpoint ด้วย CRIU
7) Client จะรับงานที่ได้รับเลือกจาก Server มาประมวลผล โดยไม่สนใจว่าจะเป็นงานใด
8) ภายใน Client จะมีการติดตั้ง Middleware, CRIU, GROMACS และ dependency อื่น ๆ ที่ทำให้ Client สามารถทำงานภายในระบบได้
9) Coordination Server จะทำการเพิ่มงานไว้ภายใน Work Queue และจัดเก็บไฟล์งานไว้ภายใน Work Server ไว้แต่แรกก่อนการทำงานของ Client
10) ในการบันทึกงาน Snapshot จะให้ความสำคัญกับ Snapshot ล่าสุดของแต่ละงานเท่านั้น (งานเดิมที่ยังค้างอยู่ใน Snapshot Server จะถูกแทนที่ด้วยอันใหม่)

3.3 การดำเนินงาน

3.3.1 การจัดเตรียมฮาร์ดแวร์และซอฟต์แวร์

ในการวิจัย สภาพแวดล้อมการประมวลผลแบบจำลองโมเลกุลแบบกระจาย จะจำลองอยู่บน Virtual environment ภายใน OpenStack โดยระบบจะมีการ Implement แบบ Volunteer Distributed System หรือการประมวลผลแบบกระจายโดยอาสาสมัคร โดยมี Clients เป็นอาสาสมัครที่จะรับงานมาประมวลผล และ Servers เป็นผู้ควบคุมการประมวลผลจะจัดเก็บทรัพยากรการประมวลผล ซึ่ง Client แต่ละเครื่องจะถูก Implement ในเน็ตเวิร์คที่ต่างกัน และจะสื่อสารไปยัง Server ผ่าน Public Internet โดย Component ต่างๆจะถูกสร้างขึ้นดังนี้



ภาพที่ 3.3-1 Network Topology ของระบบภายใน OpenStack

3.3.1.1 Client

คอมพิวเตอร์ Client จะเป็นเครื่องคอมพิวเตอร์ที่จำลองเป็นอาสาสมัครของระบบการประมวลผลแบบกระจาย ในงานวิจัยนี้จะกำหนด Client ทั้งหมด 3 เครื่อง โดยแต่ละเครื่องจะมีรายละเอียดทางเทคนิคดังนี้:

- Hardware Spec:
    - OS: Ubuntu 24.04 LTS (Minimal),
    - CPU: 4 vCPUs (Intel Xeon E3-12xx v2)
    - RAM: 8 GB
    - Volume: 50 GB
- Software Ecosystem:
    - Middleware: โปรแกรมจัดการการทำงานภายใน Client เขียนด้วยภาษา C++20 โดยใช้ Library สำคัญได้แก่:
        - Boost (Asio, Beast, Process, JSON, Log, Chrono) 1.90: สำหรับการติดต่อสื่อสารไปยัง Server ด้วย HTTP Protocol และการจดบันทึก Log
        - ActiveMQ-CPP (CMS API) 3.9.5: สำหรับการเชื่อมต่อกับ Message Broker
        - libcriu: C++ CRIU library สำหรับการทำ Checkpoint/Restore ระดับ User-space
    - GROMACS 2026.0: โปรแกรมหลักสำหรับการประมวลผลแบบจำลองโมเลกุล
    - CRIU 4.2 "CRIUTIBILITY": เครื่องมือหลักในการจัดการสถานะของ Process การทำ Checkpoint/Restore ระดับ User-space
    - pigz 2.8: ใช้สำหรับการบีบอัดข้อมูล Snapshot

3.3.1.2 Server

Server จะประกอบไปด้วย Coordination Server ทำหน้าที่เป็นตัวกลางสำหรับการติดต่อสื่อสารระหว่าง Client และ Server ในการกระจายงานการประมวลผล จัดการการบันทึก Snapshot ผลลัพธ์ และจัดการ Work Queue และ Storage Servers จำนวน 3 หน่วย (Work Server, Snapshot Server, Result Server) สำหรับการจัดเก็บทรัพยากรการประมวลผล

- Hardware Spec:

- OS: Ubuntu 24.04 LTS (Minimal),
- CPU: 4 vCPUs (Intel Xeon E3-12xx v2)
- RAM: 8 GB
- Volume: 100 GB

- Software Ecosystem:
  - Coordination Server
    - Middleware: โปรแกรมจัดการการทำงานภายใน Coordination Server เขียนด้วยภาษา C++20 โดยใช้ Library สำคัญได้แก่:
      - Boost (Asio, Beast, JSON, UUID) 1.90: สำหรับการติดต่อสื่อสารไปยัง Client ด้วย HTTP Protocol และการสร้าง REST API (Port 65096) เพื่อรับ-ส่งข้อมูลไฟล์และจัดการ Metadata ของงาน และการกำหนดชื่อไฟล์ให้ unique
      - ActiveMQ-CPP (CMS API) 3.9.5: สำหรับการเชื่อมต่อกับ Message Broker
      - MinIO C++ SDK: C++ MinIO Client สำหรับการติดต่อไปยัง Storage Server เพื่อรับ/ส่งไฟล์
    - Apache Artemis 2.53.0: โปรแกรม Message Broker สำหรับการจัดการ Work Queue และ Snapshot Queue
  - Storage Server (Work Server, Snapshot Server, Result Server)
    - MinIO: Object Storage ใช้สำหรับจัดเก็บไฟล์โครงสร้างโมเลกุล (.tpr) ความคืบหน้าการประมวลผล (Snapshot: CRIU Images) และไฟล์ผลลัพธ์สุดท้ายของการประมวลผล (.gro, .log, .edr, .cpt)

3.3.2 ขั้นตอนการทำงาน

3.3.2.1 การจัดการคำของาน

Coordination Server ได้มีการออกแบบ Message Broker ด้วย Apache Artemis ซึ่งประกอบด้วย Work Queue สำหรับการจัดเรียงงานต้นฉบับ และ Snapshot Queue เป็นพื้นที่จัดเก็บงานที่เป็น Snapshot ชั่วคราว ซึ่ง Work Queue จะมีการจัดลำดับความสำคัญ โดยระบบจะกำหนดค่า Priority ของงานประเภท Snapshot ไว้ที่ระดับ 6 ซึ่งเป็นระดับที่สูงกว่างาน

ประเภท Topology ที่ถูกตั้งค่าไว้ที่ระดับ 5 ส่งผลให้เมื่อ Client ทำการร้องขอข้อมูล Message Broker จะพิจารณาจัดสรรความคืบหน้าการประมวลผล (Snapshot) ให้เป็นลำดับแรก

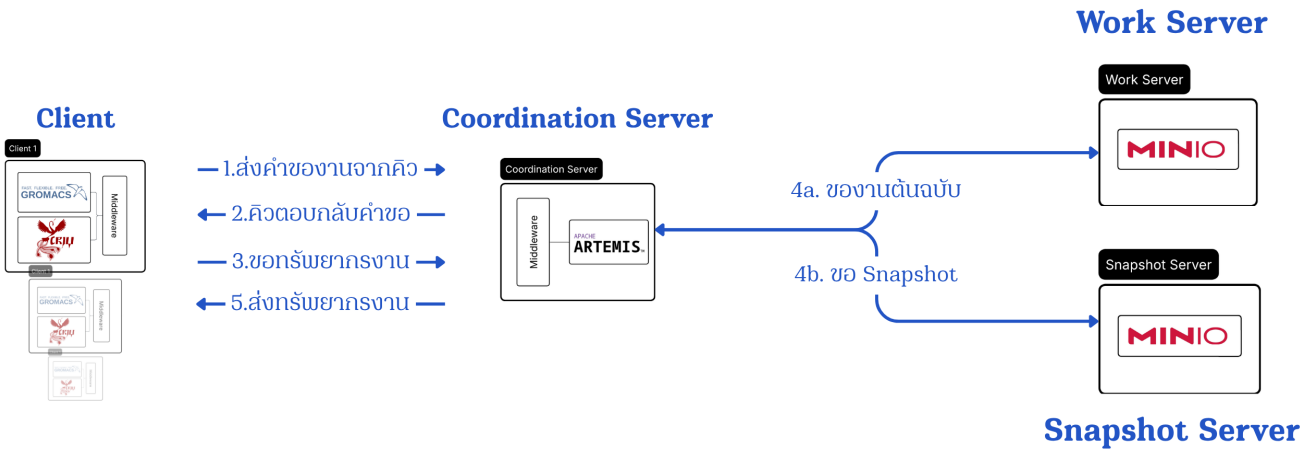
นอกจากนี้ ภายใน Coordination Server จะมีการออกแบบ REST API ไว้ภายใน Middleware เพื่อให้ Client สามารถติดต่อได้ ประกอบไปด้วย

- /get_work สำหรับการขอรับงาน
- /send_snapshot การขอส่งไฟล์ Snapshot และ
- /send_result การขอส่งไฟล์ผลลัพธ์

ในภาพที่ 3.3-2 เมื่อ Middleware บนเครื่อง Client เริ่มทำงาน จะดำเนินการดึงข้อมูลงาน (Dequeue) จาก Message Broker ที่อยู่ภายใน Coordination Server เมื่อได้รับข้อความแล้ว Client จะส่งคำร้องขอผ่าน /get_work ไปยัง Coordination Server พร้อมทั้งแนบข้อมูลฮาร์ดแวร์ (เช่น จำนวน CPU Core, รุ่นของระบบปฏิบัติการผ่านไลบรารี hwinfo) เพื่อดำเนินการดาวน์โหลดข้อมูลมาจัดเก็บไว้ในพื้นที่ชั่วคราว (/var/tmp/)

3.3.2.2 การจำแนกประเภทภาระงานเพื่อการประมวลผล

ในภาพที่ 3.3-2 เมื่อ Coordination Server ได้รับการร้องขอผ่าน /get_work ระบบจะตรวจสอบประเภทของงานที่แนบมากับ Request เพื่อพิจารณาการตอบกลับ กรณีเป็นงานประเภท Snapshot จะดาวน์โหลด Snapshot จาก Snapshot Server และทำการตอบกลับไปยัง Client เพื่อการทำงานต่อจากความคืบหน้าเดิม แต่หากกรณีเป็นงานประเภท Topology จะดึงไฟล์งานจาก Work Server และทำการตอบกลับไปยัง Client เพื่อนำไปใช้ในการรันการประมวลผลต่อไป

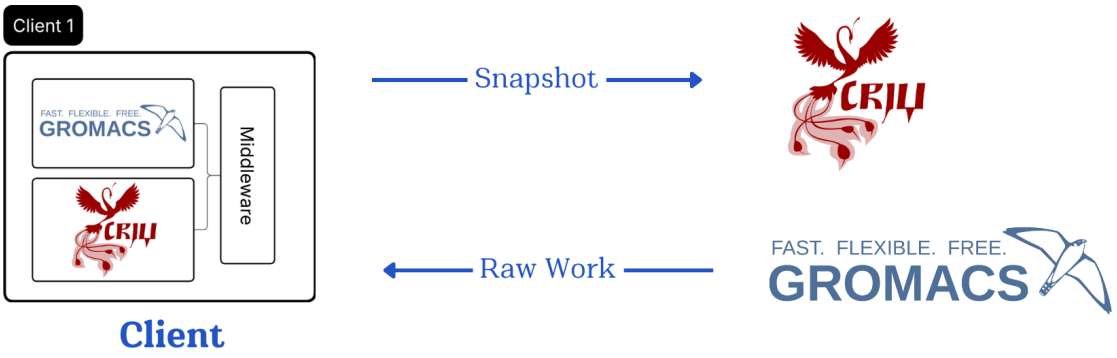


ภาพที่ 3.3-2 แผนผังขั้นตอนการจัดการคำของานระหว่าง Client และ Server

3.3.2.3 กระบวนการประมวลผล

ในภาพที่ 3.3-3 ภายหลังจากการดาวน์โหลดข้อมูลสำเร็จ Client จะตรวจสอบประเภทของงานเพื่อพิจารณาการทำงาน ในกรณีที่เป็นไฟล์ Topology จะทำการสร้าง Subprocess และใช้คำสั่งรันการประมวลผลไฟล์ Topology ด้วย GROMACS (gmx mdrun)

แต่กรณีเป็นการคืนค่าความคืบหน้าจาก Snapshot จะเรียกใช้ฟังก์ชันเพื่อ extract tarball ไฟล์ .tar.gz จากนั้น ระบบจะอ่านข้อมูลจากไฟล์ dump.log และใช้ Regular Expression ในการค้นหาค่า Process ID (PID) เดิม ก่อนที่จะเรียกใช้ฟังก์ชัน criu_restore_child (กำหนดให้เป็น Child Process ของ Middleware เช่นเดิม) คืนสถานะของโปรแกรม GROMACS ให้กลับมาดำเนินกระบวนการประมวลผล

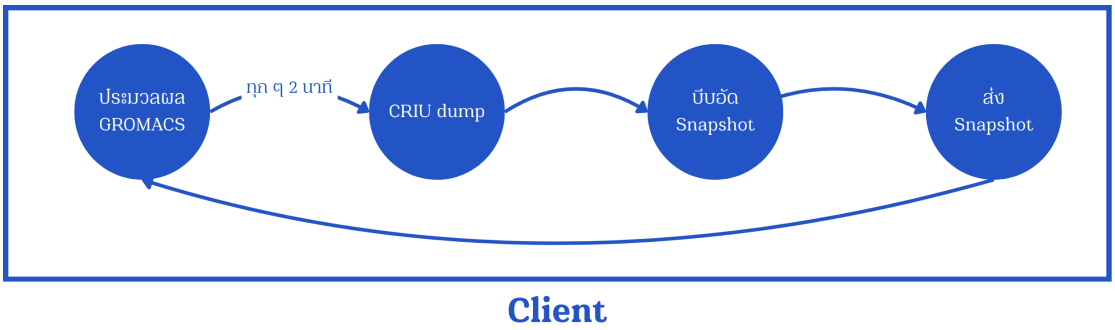


ภาพที่ 3.3-3 แผนผังการรันการประมวลผลตามประเภทงานของ Client

3.3.2.4 การบันทึกความคืบหน้าโดยอัตโนมัติ (Periodic Checkpoint)

ในภาพที่ 3.3-4 Middleware กำหนดระยะเวลาเพื่อทำ Periodic Checkpoint (ในระบบกำหนดไว้ที่ 2 นาที) เมื่อครบรอบระยะเวลาที่กำหนด ระบบจะเรียกฟังก์ชัน criu dump เพื่อใช้ CRIU ในการทำ checkpoint เพื่อบันทึกความคืบหน้าโปรแกรมลงสู่ Disk โดยจะมีการส่งพารามิเตอร์ leave_running เป็น true เพื่อให้การ Checkpoint เสร็จสิ้นแต่ยังให้การคำนวณของโปรแกรม GROMACS ยังทำงานได้ต่อภายหลังการบันทึกสถานะ เมื่อบันทึก Snapshot สำเร็จ

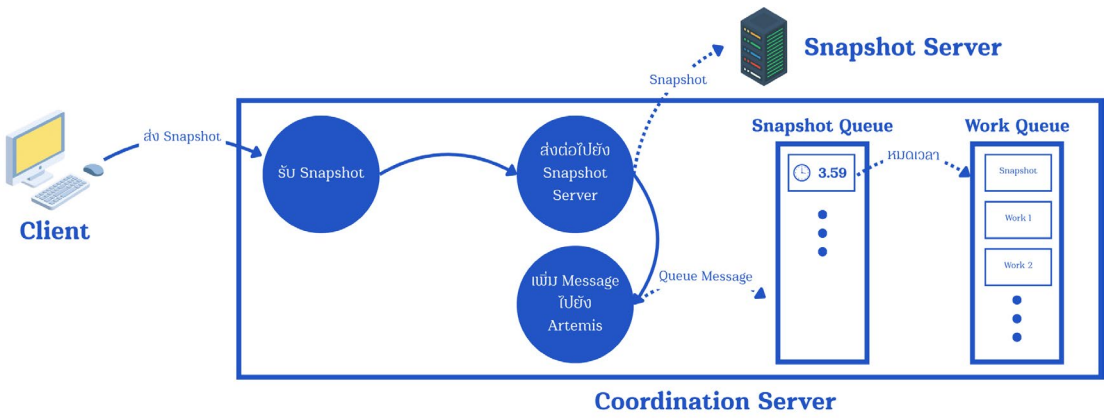
Client จะใช้ pigz บีบอัดข้อมูล Snapshot และจัดส่งไปยัง Server ผ่าน /send_snapshot พร้อมกับระบุส่วน Header ว่าการเป็นการบันทึกอัตโนมัติ



ภาพที่ 3.3-4 แผนผังการบันทึกความคืบหน้าอัตโนมัติภายใน Middleware ของ Client

ในภาพที่ 3.3-5 เมื่อ Server ได้รับ Snapshot ตัว Middleware ภายใน Coordination Server จะดำเนินการอัปโหลดข้อมูลไปยัง MinIO ภายใน Snapshot Server จากนั้น Server จะสร้างข้อความเพื่อส่งเข้าสู่ Snapshot Queue ภายใน Apache Artemis โดยกำหนดระยะเวลาที่ Message จะ Expire และย้ายไปยัง Work Queue หลัก (Client ต้องส่ง Snapshot ครั้งถัดไปภายในระยะเวลาที่กำหนด (ในระบบกำหนด 4 นาที) เพื่อแทน Heartbeat ของ Client)

หาก Client ไม่ส่ง Snapshot ภายในระยะเวลาที่กำหนดจะถือว่า Client เกิดความล้มเหลว และทำการย้าย Expired Message จาก Snapshot Queue ไปยัง Work Queue เพื่อให้ Client อื่นสามารถรับงานไปทำต่อ



ภาพที่ 3.3-5 แผนผังการบันทึกความคืบหน้าอัตโนมัติภายใน Middleware ของ Server

3.3.2.5 การส่งผลลัพธ์ภายหลังการประมวลผล

Middleware จะตรวจสอบสถานะของ GROMACS ผ่าน waitpid(gromacs_pid, &status, WNOHANG) ซึ่งเป็นการตรวจสอบแบบ Non-blocking ว่ายังมี PID ของ GROMACS อยู่หรือไม่

เมื่อคำสั่ง waitpid ตรวจพบสัญญาณการสิ้นสุดกระบวนการของโปรแกรม GROMACS Client จะดำเนินการรวบรวมผลลัพธ์ (ได้แก่ .cpt, .edr, .gro, .log) ทำการบีบอัด และส่งคำร้องขอ /send_result ไปยัง Server เพื่อส่งผลลัพธ์

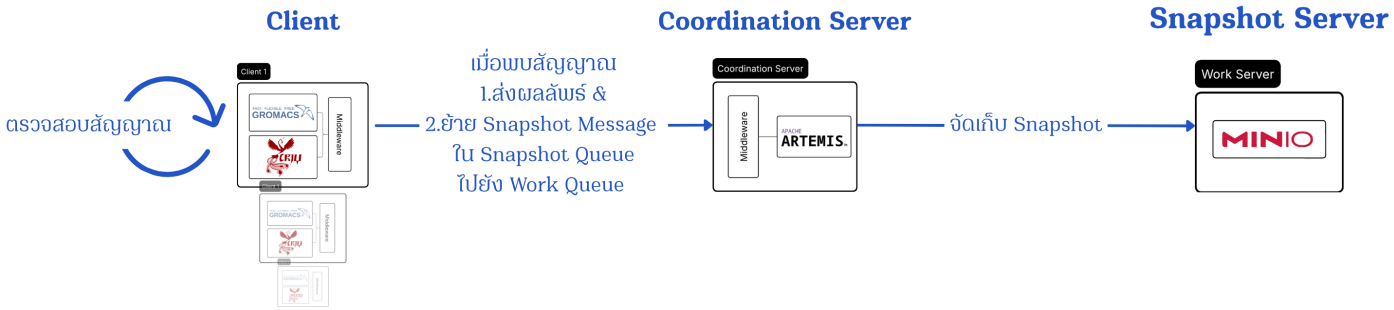
ฝั่ง Server เพื่อป้องกันความซ้ำซ้อนของข้อมูล ระบบจะเรียกใช้ฟังก์ชันตรวจสอบไฟล์ของ MinIO เพื่อตรวจประเมินว่ามีไฟล์ผลลัพธ์ปรากฏอยู่แล้วหรือไม่ หากตรวจพบ ระบบจะ reject การส่ง แต่หากไม่พบ ระบบจึงจะดำเนินการบันทึกข้อมูลลงสู่หน่วยจัดเก็บ results อย่างสมบูรณ์ ในลำดับสุดท้าย Client ทำการ mark ข้อความในคิวที่มี _AMQ_LVQ_NAME ที่ชื่อตรงกับไฟล์ข้อมูลที่เพิ่งทำเสร็จว่า finished snapshot เพื่อกำหนดว่า Snapshot ของชื่อไฟล์นี้จะไม่ใช้งานแล้วเนื่องจากงานสำเร็จแล้ว

3.3.2.6 การจัดการความล้มเหลว

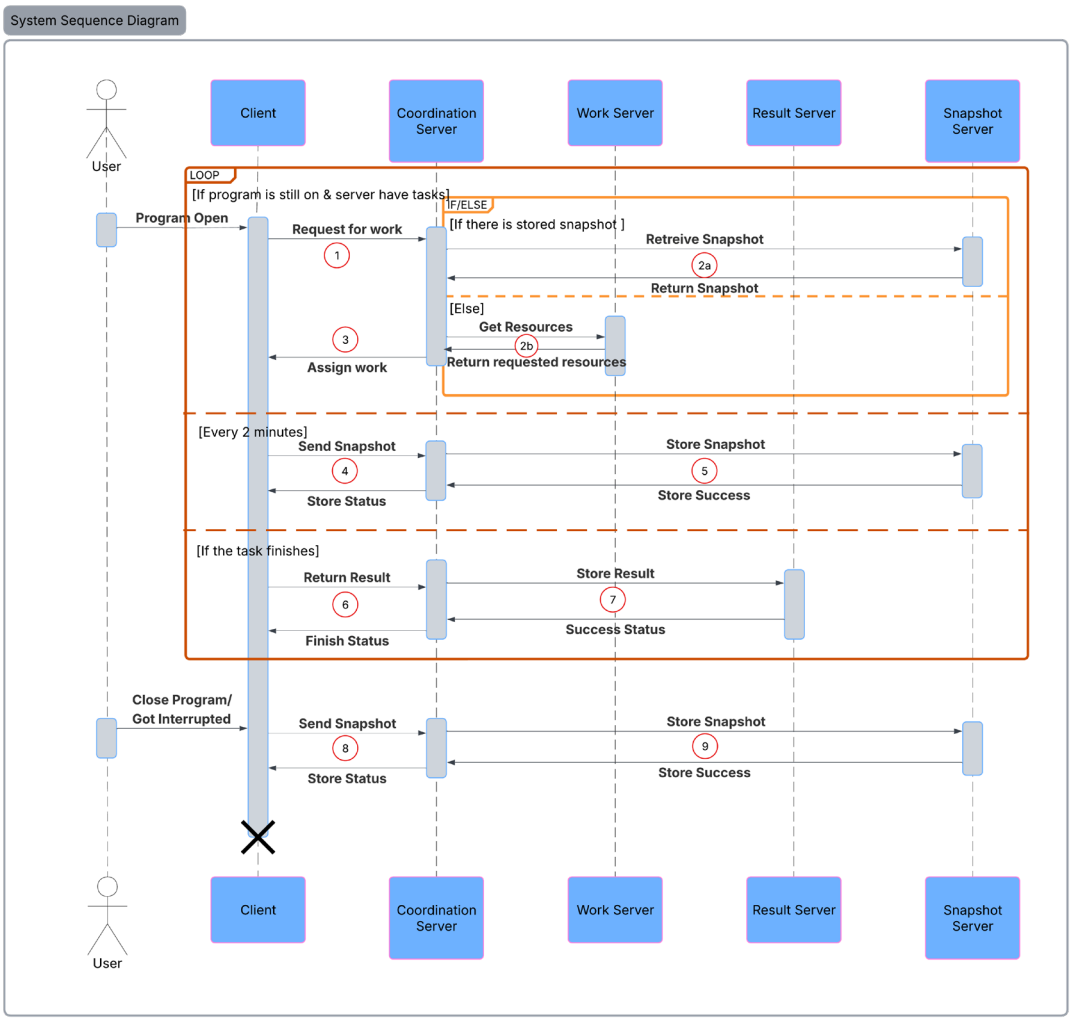
ในภาพที่ 3.3-6 Middleware ใน Client จะมีการ Setup โปรแกรมเพื่อให้คอยตรวจสอบสัญญาณ SIGINT และ SIGTERM และกำหนด custom action สำหรับการจัดการสัญญาณ ในระหว่างที่กระบวนการ GROMACS กำลังดำเนินการ

ในสถานการณ์ที่ Client ถูกบังคับหยุดการทำงาน (เช่น การปิดเครื่อง หรือการสั่งการจากระบบปฏิบัติการ) ระบบได้ออกแบบกลไกการดักจับสัญญาณ (Signal Handler) สำหรับ SIGINT และ SIGTERM ไว้ล่วงหน้า เมื่อเกิดสถานการณ์ดังกล่าว Middleware จะหยุดการทำงานในปัจจุบันและเรียกใช้ฟังก์ชัน Checkpoint ในทันที โดยการเรียกใช้ในบริบทนี้จะละเว้นพารามิเตอร์ leave_running (เพื่อให้โปรแกรมหยุดการทำงาน) เมื่อบันทึก Snapshot สำเร็จ Client จะใช้ pigz บีบอัดข้อมูล Snapshot และจัดส่งไปยัง Server ผ่าน /send_snapshot พร้อมกับระบุส่วน Header ว่าการเป็นการบันทึกจากความล้มเหลว (ไม่ใช่การบันทึกอัตโนมัติ) ภายหลังจากการบันทึกสถานะและนำส่งไปยัง Server ระบบ Server จะทำการประเมิน Header ว่าเป็นการบันทึกอัตโนมัติหรือไม่ ซึ่งในกรณีนี้จะเป็น false Server จึงจะออกคำสั่ง เพื่อบังคับให้งานใน Snapshot Queue ที่มีชื่อตรงกันหมดอายุในทันที (จากที่เคยส่งด้วย Periodic Snapshot) และส่งต่อไปยัง Work Queue ส่งผลให้ Client สามารถนำความคืบหน้าการประมวลผลดังกล่าวไปดำเนินการต่อได้

ในกรณีที่ระบบเกิดพบข้อผิดพลาดในการอัปโหลด Middleware ได้กำหนดฟังก์ชันให้ทดลองใหม่อีกครั้งซึ่งอาศัยอัลกอริทึม Exponential Backoff ในการหน่วงเวลาและพยายามส่งต่อข้อมูลซ้ำ



ภาพที่ 3.3-6 แผนภาพการจัดการความล้มเหลวของ Client และการบันทึกไปยัง Server



ภาพที่ 3.3-7 ภาพการทำงานของระบบระหว่าง Client และ Server

บทที่ 4
ผลการดำเนินงาน

การออกแบบระบบการประมวลผลแบบจำลองโมเลกุลแบบกระจายด้วย GROMACS ภายใต้การออกแบบการป้องกันความคืบหน้าการประมวลผลจากความล้มเหลว โดยการประยุกต์ใช้ระบบ Checkpoint/Restore In Userspace (CRIU) ในการย้ายความคืบหน้าการประมวลผลไปยัง Client อื่นภายในระบบผ่าน Server ได้มีการวัดผลการดำเนินงานที่ออกแบบตามวิธีการดำเนินงานในบทที่ 3 โดยมีรายละเอียดผลลัพธ์ดังต่อไปนี้

4.1 ผลการประเมินประสิทธิภาพ

ในระหว่างการประมวลผล ไฟล์โครงสร้างการประมวลผลแบบจำลองโมเลกุล 3 ประเภท ซึ่งมีความแตกต่างกันในด้านขนาดและความซับซ้อน ได้แก่ ligandCMET (แบบจำลองขนาดเล็ก) benchMEM (แบบจำลองขนาดกลาง) และ benchRIB (แบบจำลองขนาดใหญ่และมีความซับซ้อนสูง) ได้มีการวัดและบันทึกประสิทธิภาพการประมวลผลของแต่ละไฟล์ ซึ่งได้ผลลัพธ์ต่าง ๆ ดังปรากฏตารางที่ 4.1 และตารางที่ 4.2

ตารางที่ 4-1 การประเมินประสิทธิภาพของการประมวลผลแบบจำลองโมเลกุล

Topology	Topology Size (MB)	# Atom	Avg Save TPR (ms)	Result Size (MB)	Avg Upload Res (ms)	Performance (ns/day)
ligandCMET	0.184	6,443	26.8	0.314	1.83	13.278
benchMEM	3.381	81,743	94	3.816	27	5.813
benchRIB	116.933	2,136,412	1826.750	99.859	1197	0.197

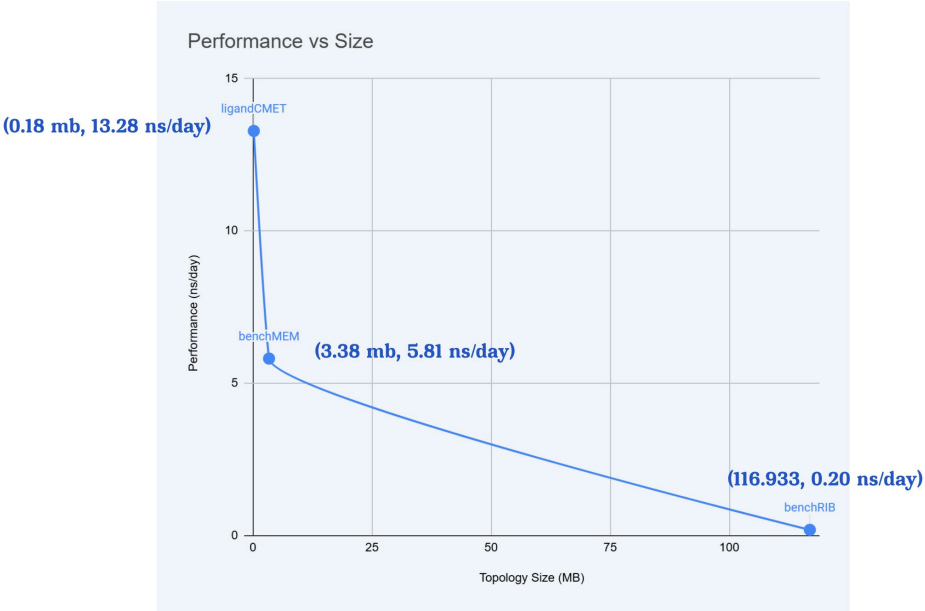
ตารางที่ 4-2 การประเมินประสิทธิภาพของการบันทึกความคืบหน้าการประมวลผลแบบจำลองโมเลกุล

Topology	Snapshot Size (MB)	Avg Save Snapshot (ms)	Avg Checkpoint (ms)	Avg Compression (ms)	Avg Upload CP (ms)	Avg Restore (ms)
ligandCMET	6.63	274	76.67	122.67	61.33	57
benchMEM	46.9	984.75	302.9	1326.8	568.7	267.25
benchRIB	1100	17629	4779.78	32770.22	69103	4187

4.2 บทสรุปและการวิเคราะห์ผลการดำเนินงาน

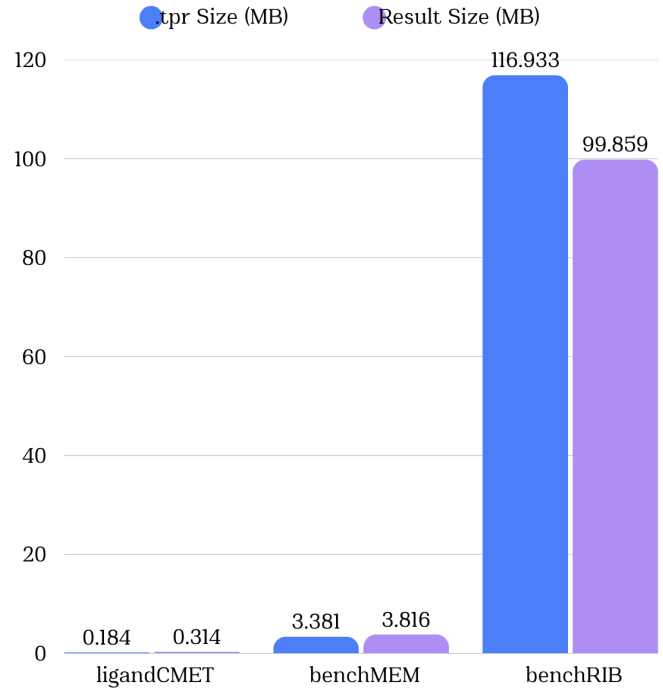
จากการวิเคราะห์ข้อมูลในตารางสรุปผลวิจัย สามารถอธิบายรายละเอียดและข้อค้นพบที่สำคัญของการทดลองใช้งานระบบการประมวลผลแบบจำลองโมเลกุลแบบกระจายที่ประยุกต์ใช้ Checkpoint/Restore ด้วย CRIU ดังนี้

1) อิทธิพลของขนาดไฟล์โครงสร้างแบบจำลองโมเลกุลต่อประสิทธิภาพโดยรวม: ขนาดของงานจำลอง (วัดจาก Atom Count และ Topology Size) ส่งผลโดยตรงต่อทั้งประสิทธิภาพการประมวลผลและระยะเวลาในการจัดการความล้มเหลว:
   a) ประสิทธิภาพการประมวลผล: จากตาราง 4.2 พบว่าประสิทธิภาพการคำนวณ (Performance (ns/day)) ลดลงอย่างมากเมื่อขนาดโมเดลเพิ่มขึ้น โดยงานจำลองขนาดเล็กสุด (ligandCMET) มีประสิทธิภาพ 13 ns/day และงานจำลองขนาดกลาง (benchMEM) มีประสิทธิภาพ 5.813 ns/day ในขณะที่งานจำลองขนาดใหญ่สุด (benchRIB ซึ่งมีจำนวนอะตอมมากกว่า 2.1 ล้านอะตอม) มีประสิทธิภาพที่ต่ำมาก (0.197 ns/day) ซึ่งชี้ให้เห็นว่างานขนาดใหญ่มีความต้องการทรัพยากรการประมวลผลสูงมากเมื่อเทียบกับเวลาที่ใช้



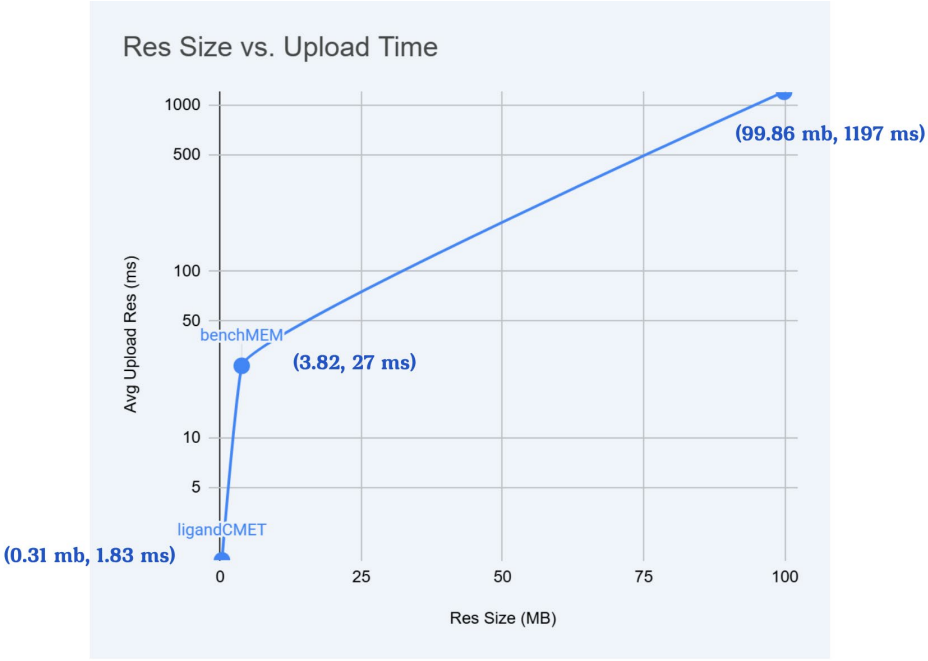
ภาพที่ 4.2-2 แผนภูมิเส้นแสดงขนาดของไฟล์งานและประสิทธิภาพการประมวลผล

b) ขนาด Snapshot: ขนาดไฟล์ Snapshot ที่ต้องจัดเก็บ (Snapshot Size (MB)) มีความสัมพันธ์โดยตรงกับจำนวนอะตอม โดย benchRIB สร้าง Snapshot ที่มีขนาดใหญ่ถึง 1.1 GB ซึ่งเป็นปัจจัยสำคัญในการจัดการการบันทึกและการส่งต่อสถานะ



ภาพที่ 4.2-1 แผนภูมิแท่งแสดงขนาดของไฟล์งาน (.tpr) และขนาดของผลลัพธ์

c) เวลาอัปโหลดผลลัพธ์ (Avg Upload Res (ms)): การอัปโหลดผลลัพธ์สุดท้ายของทุกไฟล์ใช้เวลาที่ค่อนข้างน้อย 1.830 ms 27 ms และ 1197 ms สำหรับ ligandCMET benchMEM และ benchRIB ตามลำดับ



ภาพที่ 4.2-3 แผนภูมิเส้นแสดงขนาดของผลลัพธ์และระยะเวลาอัปโหลดผลลัพธ์

2) ประสิทธิภาพของการดำเนินการ Checkpoint: ในกระบวนการของการบันทึกสถานะ ระยะเวลาการ Checkpoint การบีบอัด Snapshot และการ Upload Snapshot ถือเป็นตัวแปรสำคัญในการกำหนดความเป็นไปได้ของการ Implement ระบบให้มีกระบวนการ Checkpoint/Restore

a) ระยะเวลาของแต่ละกระบวนการ

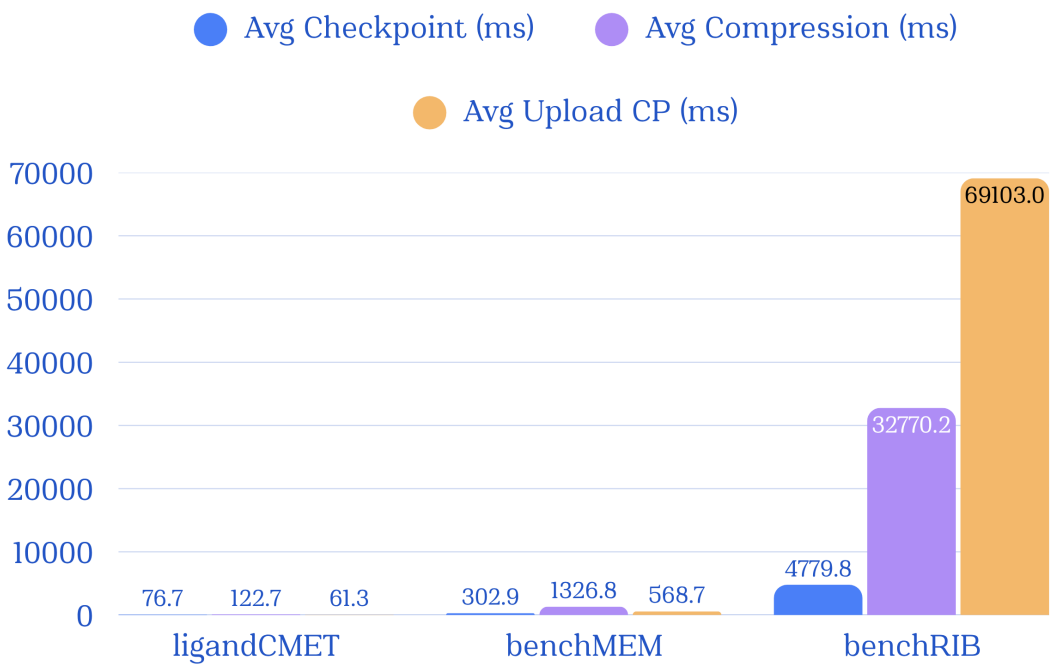
i) เวลา Checkpoint: เวลาเฉลี่ยในการสร้าง Checkpoint (Avg Checkpoint (ms)) ในขณะที่ที่ ligandCMET และ benchMEM ใช้เวลาเพียง 76.67 ms และ 302.9 ms ซึ่งเป็นเวลาที่ค่อนข้างน้อยจนไม่เป็นที่สังเกต แต่ benchRIB ใช้เวลาถึง 4,779.78 ms (ประมาณ 4.8 วินาที สำหรับ benchRIB) ซึ่งเพิ่มขึ้นอย่างมาก

ii) การบีบอัดข้อมูล (Avg Compression (ms)): การบีบอัด Snapshot มีการใช้เวลาสูงถึง 32,770.22 ms (ประมาณ 32.8 วินาที) ซึ่งมาจากการบีบอัด CRIU Image ที่ค่อนข้างใหญ่ (1.1 GB)

iii) การอัปโหลด Checkpoint (Avg Upload CP (ms)): เป็นจุดที่ส่งผลกระทบที่ใหญ่ที่สุด โดย benchRIB ใช้เวลาในการอัปโหลดสูงถึง 69,103 ms (ประมาณ 1.15 นาที) ซึ่งกิน

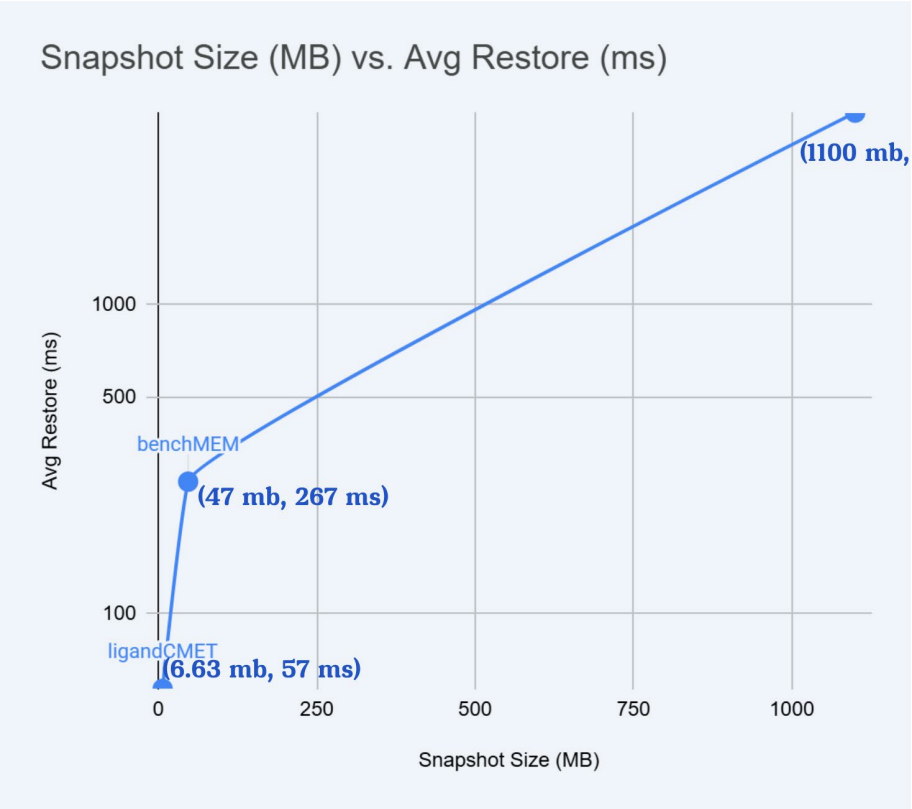
ส่วนแบ่งเวลาส่วนใหญ่ของช่วงเวลา Periodic Checkpoint (ที่ถูกกำหนดไว้ 2 นาที) การอัปโหลดที่ใช้เวลานานนี้อาจบ่งชี้ถึงข้อจำกัดที่อาจทำให้ไม่คุ้มค่ากับการประมวลผลไฟล์ขนาดใหญ่

b) ระยะเวลารวมที่ใช้ในการบันทึกความคืบหน้าการประมวลผล (ระยะเวลาการ Checkpoint การบีบอัด Snapshot และการ Upload Snapshot) สำหรับ ligandCMET benchMEM และ benchRIB จะอยู่ที่ประมาณ 0.27 วินาที 2.2 วินาที และ 106.65 วินาทีตามลำดับ ซึ่งหากประเมินจากที่ Ubuntu OS อนุญาตให้โปรแกรมหยุดทำงานก่อนการ Shutdown ภายใน 90 วินาที สำหรับ ligandCMET และ benchMEM สามารถบันทึกได้รวดเร็วและทันเวลา แต่สำหรับ benchRIB อาจไม่สามารถบันทึกได้ทันเวลา ด้วยระยะเวลาการบีบอัดและอัปโหลดที่สูงมาก เนื่องจากไฟล์ Snapshot ที่ใหญ่ตามขนาดของ Topology file



ภาพที่ 4.2-4 แผนภูมิแท่งแสดงระยะเวลาเฉลี่ยในแต่ละกระบวนการบันทึกความคืบหน้า (Checkpoint, Compression และ Upload) ของการประมวลผลไฟล์ ligandCMET benchMEM และ benchRIB

3) เวลา Restore: เวลาเฉลี่ยในการคืนค่าสถานะ (Avg Restore (ms)) มีแนวโน้มเพิ่มขึ้นเช่นกัน โดยใช้เวลา 57 ms 267.25 ms และ 4,187 ms (หรือประมาณ 4.2 วินาที) สำหรับ ligandCMET benchMEM และ benchRIB ตามลำดับ ซึ่งใกล้เคียงกับเวลาที่ใช้ในการ Checkpoint เมื่อเปรียบเทียบกับขั้นตอนอื่นๆ (เช่น การบีบอัดและการอัปโหลด) การคืนค่าสถานะด้วย CRIU เองถือว่าค่อนข้างรวดเร็วและไม่ส่งผลกระทบต่อการทำงานภาพรวมสำหรับงานขนาดใหญ่ เมื่อไฟล์ Snapshot ถูกดาวน์โหลดแล้ว



ภาพที่ 4.2-5 แผนภูมิเส้นแสดงขนาดของ Snapshot และระยะเวลาการ Restore

บทที่ 5
สรุป

โครงงานนี้สามารถนำเสนอแนวทางการประยุกต์ใช้ Checkpoint/Restore ด้วย CRIU เพื่อแก้ไขปัญหาความล่าช้าและการสูญเสียความคืบหน้าในระบบการประมวลผลแบบจำลองโมเลกุลแบบกระจาย โดยเฉพาะในสภาพแวดล้อมแบบ Volunteer Computing ที่อาสาสมัครอาจไม่สามารถเปิดเครื่องประมวลผลได้ตลอดเวลาหรืออาจเกิดความล้มเหลวของระบบได้ทุกเมื่อ ระบบที่พัฒนาขึ้นประกอบด้วยส่วนของ Client ที่ใช้ GROMACS เป็นโปรแกรมหลักในการประมวลผลแบบจำลองโมเลกุล CRIU สำหรับการบันทึกสถานะรวมไปถึงการคืนสถานะของการประมวลผล โดยทำงานร่วมกันผ่าน Middleware ที่ออกมาเพื่อควบคุมการทำงานของ Client ในระบบการประมวลผลแบบกระจาย และส่วนของ Server ที่ทำหน้าที่เป็นศูนย์กลางการประมวลผล โดยมี Coordination Server ที่ใช้ Apache Artemis ในการจัดการคิวงานโดยมี Middleware สำหรับการติดต่อสื่อสารกับ Client และประสานงานไปยัง Storage Servers (Work Server, Snapshot Server, Result Server) ที่ทำหน้าที่เป็นหน่วยจัดเก็บข้อมูลงานต้นฉบับ ความคืบหน้า (Snapshot) และผลลัพธ์

ในด้านการดำเนินงานของระบบ การทำงานหลักอยู่ที่การบริหารจัดการภาระงานผ่าน Work Queue และ Snapshot Queue ซึ่งมีการกำหนดระดับความสำคัญ (Priority) ให้งานประเภท Snapshot อยู่ในระดับที่สูงกว่างานต้นฉบับ เพื่อให้มั่นใจว่างานที่มีความคืบหน้าจะถูกดึงไปประมวลผลต่อก่อน ระบบถูกออกแบบให้รองรับการจัดการความล้มเหลวสองรูปแบบหลัก คือ (1) ความล้มเหลวที่จัดการได้ (Graceful Shutdown) เช่น การปิดเครื่องปกติ ซึ่ง Middleware จะดักจับสัญญาณ SIGINT/SIGTERM เพื่อทำ Final Checkpoint และ Migrate สถานะไปยัง Server ทันที และ (2) ความล้มเหลวที่จัดการไม่ได้ เช่น ระบบค้างหรือถูกตัดไฟ ซึ่งจะใช้กลไก Periodic Checkpoint ทุก 2 นาที เพื่อส่งสถานะล่าสุดเป็นระยะแทน Heartbeat ของ Client ซึ่งหากเงียบหายไปเกินเวลาที่กำหนด Server จะทำการ Promote Snapshot ล่าสุดในคิวชั่วคราว (Snapshot Queue) กลับเข้าสู่ Work Queue เพื่อให้ Client เครื่องอื่นรับช่วงต่อได้ทันทีโดยไม่สูญเสียความคืบหน้าทั้งหมดไป

จากการทดสอบประสิทธิภาพเชิงปริมาณ พบว่าระบบสามารถทำงานได้ตาม Functional Requirement อย่างครบถ้วน โดยเฉพาะกับแบบจำลองขนาดเล็กและกลางอย่าง ligandCMET และ benchMEM ที่มีระยะเวลาในการทำ Checkpoint และ Restore ต่ำ และขนาด

Snapshot ที่เล็กจนไม่ส่งผลกระทบต่อการประมวลผล อย่างไรก็ตาม สำหรับแบบจำลองขนาดใหญ่อย่าง benchRIB ที่มีจำนวนอะตอมกว่า 2 ล้านอะตอม พบว่าขนาดไฟล์ Snapshot ที่ใหญ่ถึง 1.1 GB กลายเป็นข้อจำกัดสำคัญโดยเฉพาะระยะเวลาในการบีบอัดข้อมูลและการอัปโหลด Snapshot ที่กินเวลาเกือบเท่ากับรอบการบันทึกอัตโนมัติ (ในระบบการทำลองกำหนดเป็น 2 นาที) ดังนั้น ข้อสรุปสำคัญคือแม้เทคนิคนี้จะช่วยรักษาความทนทานต่อความเสียหาย (Fault Tolerance) ได้ดีเยี่ยม แต่สำหรับงานขนาดใหญ่มาก ในระบบที่มีความล้มเหลวบ่อยครั้ง อาจต้องมีการเพิ่มประสิทธิภาพในการจัดการความคืบหน้าเพื่อให้คุ้มค่ากับการทำงานภายในระบบนี้

5.1 ข้อเสนอแนะ

สำหรับการพัฒนาและต่อยอดระบบในอนาคต จากผลการประเมินที่พบว่าการจัดการไฟล์จำลองที่มีขนาดใหญ่มาก (เช่น benchRIB) มีข้อจำกัดอย่างมากในด้านระยะเวลาการจัดการ Snapshot ดังนั้น ควรมีการพิจารณาปรับปรุงการประมวลผล โดยอาจใช้การแบ่งงานขนาดใหญ่ให้เป็นส่วนย่อย ๆ เพื่อกระจายงานที่มีขนาดเล็กลงไปยังคอมพิวเตอร์หลายเครื่องให้สามารถทำงานแบบขนานกันได้ การนำเทคนิค Incremental Checkpointing เข้ามาประยุกต์ใช้ เพื่อบันทึกเฉพาะสถานะหรือหน่วยความจำที่มีการเปลี่ยนแปลงจากรอบก่อนหน้า

ในด้านความทนทานต่อสภาพแวดล้อมการใช้งานจริง (Real-world Environment) เนื่องจากระบบปัจจุบันถูกทดสอบภายใต้ข้อสมมติฐานที่ว่าเครือข่ายอินเทอร์เน็ตมีความเสถียร การพัฒนาในระยะต่อไปจึงควรเพิ่มกลไกการจัดการความไม่เสถียรของเครือข่าย (Network Resilience) เช่น การจัดการเมื่อเกิด Network Disconnection หรือ High Latency เพื่อให้ Client สามารถเก็บสะสม Snapshot ไว้ในเครื่องชั่วคราวและส่งข้อมูลต่อเมื่อเครือข่ายกลับมาเป็นปกติได้ นอกจากนี้ควรพิจารณาพัฒนาระบบให้สามารถปรับรอบระยะเวลาการบันทึกความคืบหน้าอัตโนมัติ (Dynamic Checkpoint Interval) ให้ยืดหยุ่นตามความสามารถของฮาร์ดแวร์ในเครื่อง Client แต่ละเครื่อง หรือตามขนาดของไฟล์แบบจำลอง เพื่อสร้างสมดุลที่ดีที่สุดระหว่างการป้องกันข้อมูลสูญหายจากการหยุดทำงานกะทันหัน และการลดภาระการทำงาน (Overhead) ที่เกินความจำเป็นต่อระบบโดยรวม

ในการวัดผล ควรมีการเปรียบเทียบ Trade-off ในด้านระยะเวลาระหว่างการใช้การบีบอัด Snapshot และอัปโหลดไปยัง Server และการอัปโหลด Snapshot โดยตรง ถึงแม้ Snapshot จะมีขนาดเล็กลงอย่างมากภายหลังการบีบอัด แต่ความคุ้มค่าในสภาพแวดล้อมที่เน็ตเวิร์คและฮาร์ดแวร์ที่ต่างกันอาจให้ผลลัพธ์ด้านความคุ้มค่าของการบีบอัด Snapshot ที่ต่างกัน

รายการอ้างอิง

[1] S. Kemp. "Digital 2025: Device Trends for 2025." DataReportal. https://datareportal.com/reports/digital-2025-device-trends-for-2025 (accessed November 22, 2025).

[2] A. Chroneos, M. J. D. Rushton, C. Jiang, and L. H. Tsoukalas, "Nuclear wasteform materials: Atomistic simulation case studies," *Journal of Nuclear Materials,* vol. 441, no. 1, pp. 29-39, 2013, doi: https://doi.org/10.1016/j.jnucmat.2013.05.012.

[3] M. v. Steen and A. S. Tanenbaum, *Distributed Systems*, 3rd ed. distributed-systems.net, 2017.

[4] LHC@home. "What is volunteer computing?" CERN. https://lhcathome.web.cern.ch/about/volunteer-computing (accessed November 22, 2025).

[5] D. Anderson. How BOINC works [Online] Available: https://github.com/BOINC/boinc

[6] D. P. Anderson, "Globally Scheduling Volunteer Computing," *Future Internet,* vol. 13, no. 9, p. 229, 2021. [Online]. Available: https://www.mdpi.com/1999-5903/13/9/229.

[7] IBM, "What is a message broker?," vol. 2026, no. April 20. [Online]. Available: https://www.ibm.com/think/topics/message-brokers

[8] D. P. Anderson, "BOINC: A Platform for Volunteer Computing," *Journal of Grid Computing,* vol. 18, no. 1, pp. 99-122, 2020/03/01 2020, doi: 10.1007/s10723-019-09497-9.

[9] A. Tošić, "Run-Time Application Migration Using Checkpoint/Restore In Userspace," *Journal of Web Engineering,* vol. 23, no. 5, pp. 735-748, 2024, doi: 10.13052/jwe1540-9589.2357.

[10] C. Kutzner. "GROMACS benchmark set." https://www.mpinat.mpg.de/grubmueller/bench (accessed May 13, 2026).

[11] *Gemini* 3 [*Large language model]*. (2026). [Online]. Available: https://gemini.google.com/

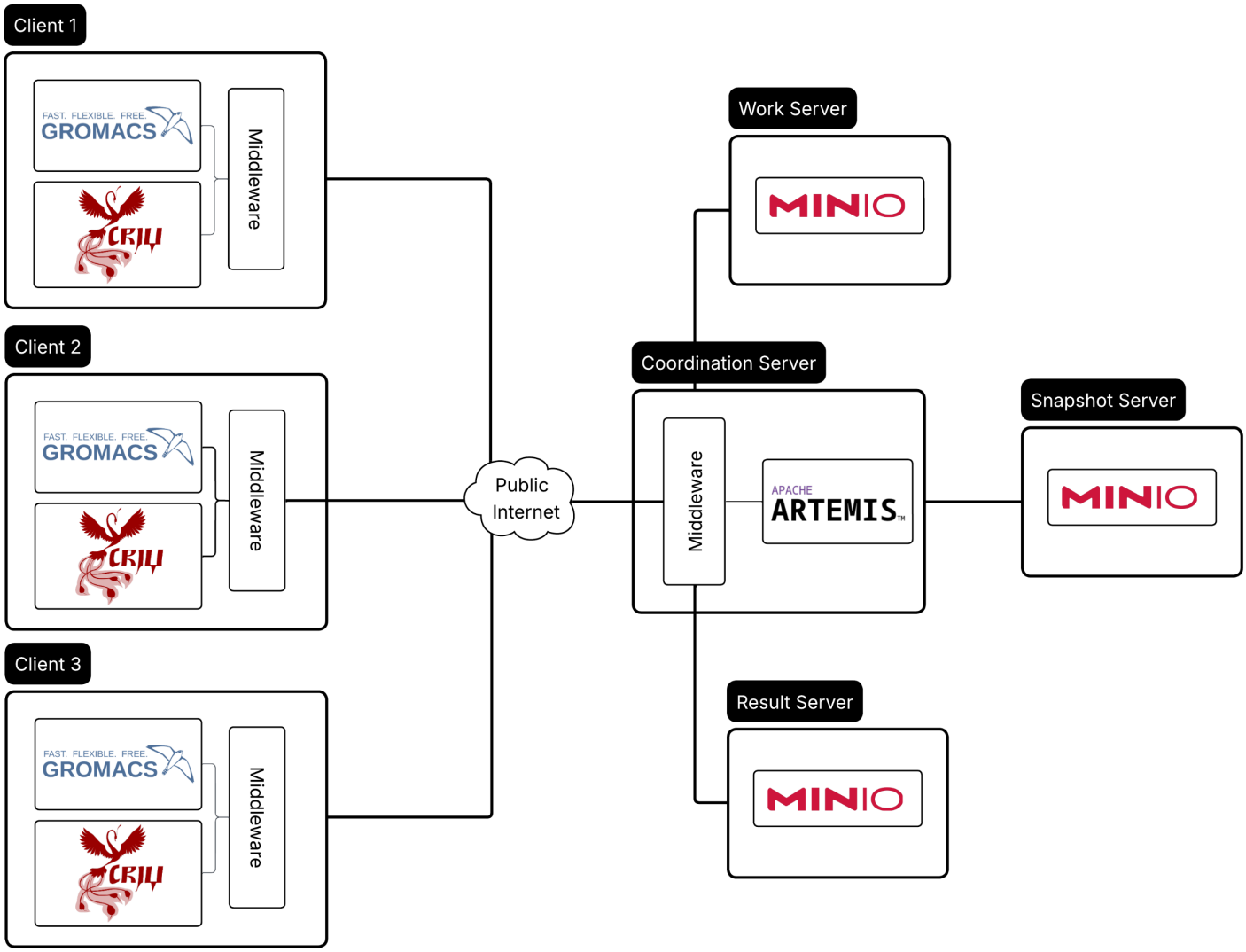
[12] *GPT*-5 [*Large language model]*. (2026). [Online]. Available: https://chatgpt.com/

* ในโครงงานนี้ ผู้วิจัยได้มีการใช้งาน Gemini 3.0 [11] และ GPT-5 [12] ในการสนับสนุนการศึกษาด้วยตนเองการประยุกต์ใช้ Checkpoint/Restore ด้วย CRIU สำหรับระบบการประมวลผลแบบจำลองโมเลกุลแบบกระจาย โดยมีการใช้งานดังนี้:

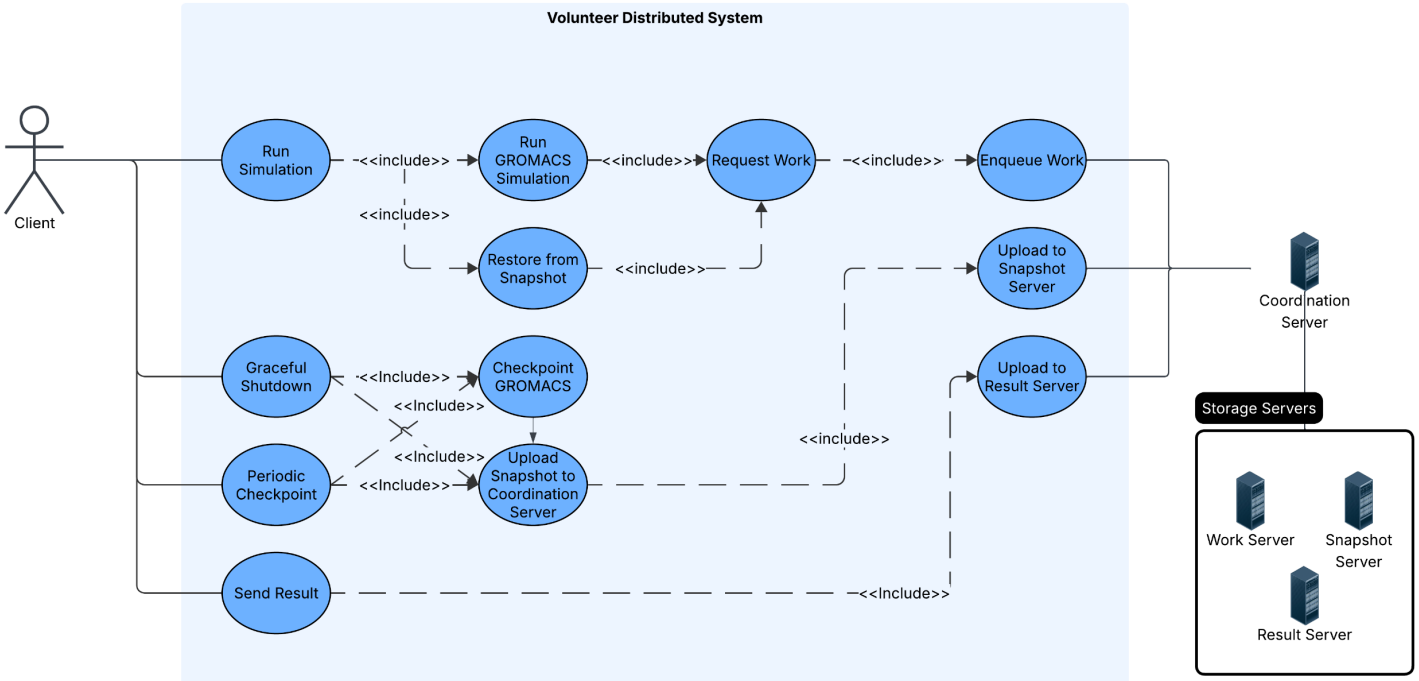
- ปรับปรุงข้อความ รูปแบบการจัดรายงานให้เหมาะสม ตรวจสอบความถูกต้องและความสมบูรณ์ของเนื้อหา
- ปรับปรุงและแก้ไขโปรแกรม Middleware ทั้งในฝั่ง Client และ Server เพื่อให้สามารถใช้งานได้และมีความถูกต้องตามแผนการศึกษาของผู้วิจัย

ภาคผนวก

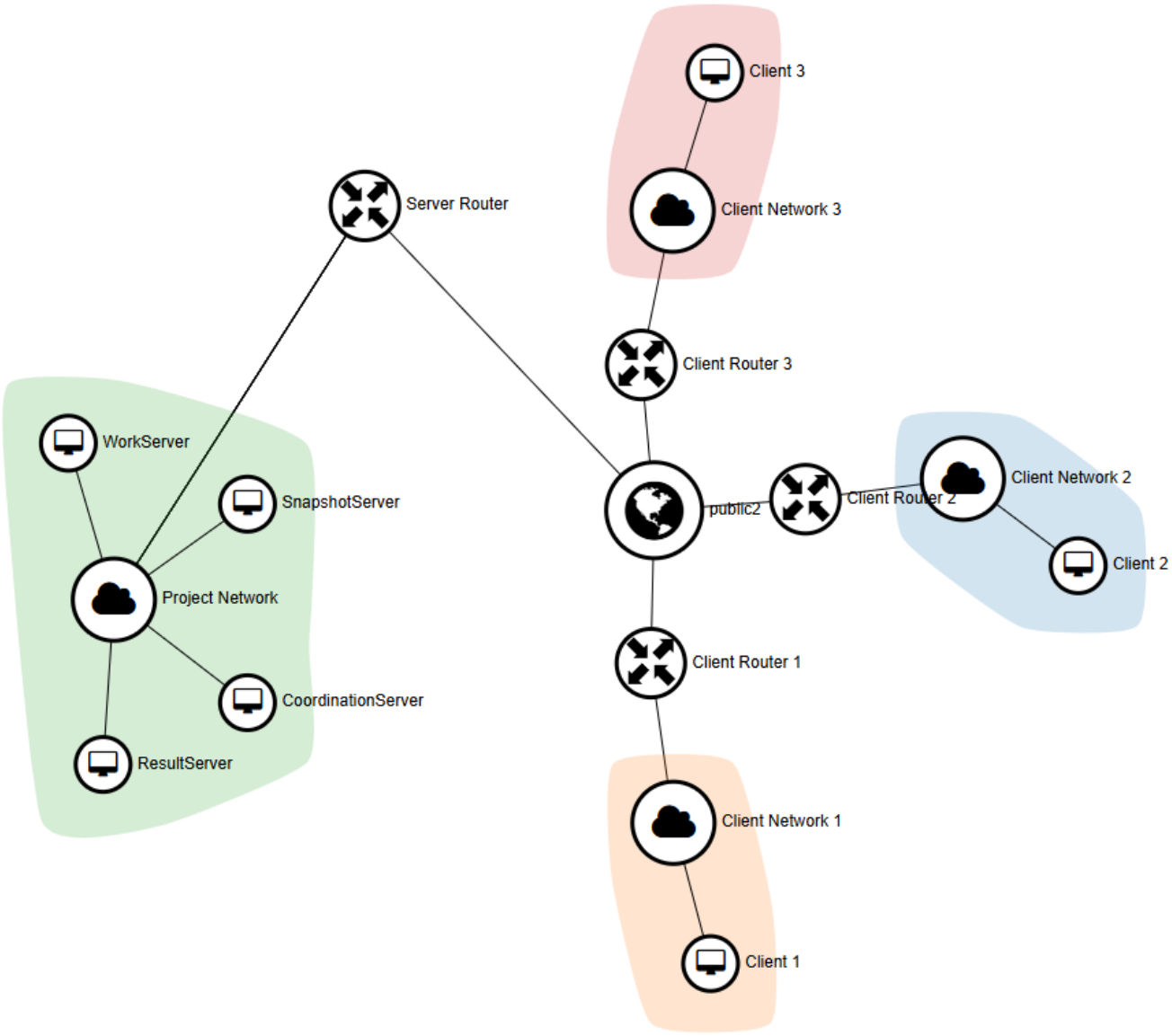
ภาคผนวก ก. ภาพโครงสร้างของระบบตัวอย่าง



ภาคผนวก ช. Use Case Diagram ของระบบ



ภาคผนวก ค. Network Topology ของระบบภายใน OpenStack



ภาคผนวก ง. ภาพการทำงานของระบบระหว่างแต่ละ Component

